

In The Name of God



Forecasting Competitive Football

Pre-Match and In-Play Prediction from Raw Event Data

Contributors: Parsa Bordbar (40435340), Navid Zare (40435071)

Data sources: StatsBomb Open Data; Football-Data.co.uk

Paper P1: Geometric SMOTE for Imbalanced Datasets with Nominal and Continuous Features

Paper P2: Hierarchical Shrinkage: Improving the Accuracy and Interpretability of Tree-Based Models

Dataset at a glance

Quantity	Value	Quantity	Value
Matches	1,517	Pre-match design columns	231
In-play snapshots	28,823	In-play design columns	385
Competitions	4 leagues, 2015/16	Train / validation / test	896 / 277 / 344
Snapshots per match	19 ($t = 0, 5, \dots, 90$)	Odds tagging coverage (test)	343 / 344 = 99.71%

Contents

1. Introduction and Problem Framing	3
2. Dataset, Integration Design and Competition Selection	6
3. Feature Engineering: Pre-Match and In-Play Pipelines	11
4. Paper Reproductions (P1 and P2)	14
5. Experimental Setup	18
6. Results: Models 1, 2 and 3	21
7. Calibration and Reliability Analysis	29
8A. Error and Failure Analysis	32
8B. In-Play Analysis	38
8C. Imbalance and Resampling	42
8D. Compute and Scaling Analysis	44
8E. Theory versus Reality.....	47
9. Conclusion and Limitations	53
A. Appendix A — Hand Derivation of the P2 Core Equation	56
B. Appendix B — Reproducibility.....	58
C. Appendix C — Complete Result Tables	61
References.....	68
List of Tables	69
List of Figures	71

1. Introduction and Problem Framing

Football is the most widely followed sport in the world, and the analytics that support it are built on event data: a complete, time-stamped record of every action on the pitch. A single match in the StatsBomb Open Data release contains on the order of four thousand such events, each carrying a type, a period, a minute, a possession index, the team and player involved, and a pitch location on a standardized 120 by 80 coordinate system. This project takes that raw material and builds a forecasting system from it.

Forecasting a football match happens at two distinct moments in practice, and the two are not the same problem. Before kick-off, a pre-match model can use only what is known about the two teams from their history: recent form, playing style, venue, and rest. This is the information set behind opening bookmaker prices. Once the match is under way, an in-play model takes over and re-estimates the same quantities continuously as goals, cards and momentum shifts arrive. This is the engine behind live betting markets and the win-probability graphics shown during broadcasts. The two models answer identical questions from completely different data pipelines, and building both without leakage is the substance of this project.

Because a forecast is judged on the quality of its stated probabilities and not merely on whether the single most likely outcome occurred, this work weights calibration and proper scoring rules heavily throughout. A model that confidently predicts a home win and is wrong is penalised more than one that expressed appropriate uncertainty. And because a forecast that cannot be explained cannot be trusted, SHAP-based explanation of individual predictions is treated as a first-class deliverable rather than an afterthought.

1.1 Formal definition of the three tasks

Each of the three deliverables is defined below by stating precisely what one input example is, what the output is, and how the training set is constructed. All three are posed over the same population of matches, so that differences between them are attributable to the information available at prediction time rather than to a change of dataset.

Model 1 — Pre-match outcome classification (Task C)

One example is one scheduled match, observed at kick-off. The feature vector is built exclusively from matches that finished strictly before that kick-off: rolling aggregates of both teams' prior performances, venue-split form, head-to-head history, rest days and the competition identifier. The output is a probability distribution over the ordered outcome set {H, D, A}, meaning home win, draw and away win. The label is derived once, from the final score recorded in the StatsBomb match file. The training set is every match whose kick-off falls inside the training block of the chronological split.

Model 2 — Pre-match goal-margin regression (Task R)

One example is the same match observed at the same moment, with the identical feature vector. The output is the signed goal margin, home minus away, clipped to the interval from minus five to plus five:

$$y_R = \text{clip}(\text{score}_{\text{home}} - \text{score}_{\text{away}}, -5, +5) \quad (1)$$

Equation (1) states the clip explicitly because it is a modelling decision rather than a data property. Large margins are rare and are driven mostly by finishing variance

once a match is already decided, so the difference between a four-goal and a seven-goal win carries very little forecasting signal but contributes a great deal of squared error. Clipping limits the influence of those matches without discarding them.

Model 3 — In-play prediction (Task L)

One example is a snapshot of one match at one regulation minute. Nineteen snapshots are taken per match, at $t = 0, 5, 10, \dots, 90$. The feature vector concatenates the frozen pre-match vector for that fixture with features computed only from the events of that same match up to and including minute t : the current score, the man-advantage state, event counts and rates, and momentum indicators over a recent ten-minute window. The output is an updated outcome distribution and an updated expected final margin, so the task splits into two internal codes, L_c for the classification label and L_r for the margin label. The label of every snapshot is its parent match's final result.

Model 3 is deliberately the hardest of the three. It requires a snapshotting scheme, a strict prohibition on any event after time t influencing the snapshot at t , a guarantee that all snapshots of one match sit on the same side of every split boundary, and calibration reasoning that is specific to the phase of the match. A direct consequence of the design is that nineteen snapshots of one match are not nineteen independent observations; they share a label and are heavily correlated. This affects cross-validation, statistical testing and error analysis, and each of those places is handled explicitly later in this report.

1.2 Evaluation metrics and why each was chosen

The headline classification metric is the ranked probability score. It is preferred to accuracy and to plain log loss because the three outcomes are ordered: a draw sits between a home win and an away win. If a model confidently predicts a home win, being wrong because the match was drawn is a smaller error than being wrong because the away team won. Accuracy cannot express that distinction at all, and log loss treats the two mistakes as equally bad. For an ordered set of K classes the score is the mean squared error of the cumulative distribution:

$$\text{RPS} = \frac{1}{K-1} \sum_{k=1}^{K-1} \left(\sum_{j \leq k} p_j - \sum_{j \leq k} o_j \right)^2 \quad (2)$$

In equation (2), p is the predicted distribution and o is the one-hot realized outcome; lower is better. Three further probabilistic measures are reported alongside it because they answer different questions. Log loss punishes confident mistakes harshly and is unbounded, which makes it sensitive to a single badly wrong prediction. The multi-class Brier score is a plain squared error on the probability vector and is bounded, so it is more stable but less discriminating. The expected calibration error asks a different question entirely: whether a stated confidence of seventy per cent is correct about seventy per cent of the time. A model can rank matches well and still be badly calibrated, so the two families of metric are not substitutes. Per-class precision, recall and F1 are reported in addition, because aggregate probabilistic scores can conceal a classifier that never predicts the minority class at all.

Regression is reported as mean absolute error, root mean squared error, and the Pearson correlation between predicted and realized margins. Mean absolute error is the headline figure because it is in the natural unit of the problem, goals. Root mean squared error is reported beside it because the gap between the two indicates how much of the total error is concentrated in a small number of large misses. Correlation is included because it separates the ability to rank matches by expected margin from the ability to state the margin on the correct scale.

1.3 Research questions

- Can a model built from event-derived features approach the accuracy of the de-vigged bookmaker market on the same matches, and if not, how large is the gap and what explains it?
- How much does live information improve a forecast relative to its own frozen pre-match prediction, and at which minutes of the match does that improvement arrive?
- Does an oversampling method designed for mixed nominal and continuous data improve minority-class detection on the draw outcome, and what does it cost in probabilistic calibration?
- Does a post-hoc tree-regularization method produce better-calibrated probabilities than tuned library baselines, and is any observed advantage statistically distinguishable from noise on a dataset of this size?

2. Dataset, Integration Design and Competition Selection

The event data come from StatsBomb Open Data and the market baseline from Football-Data.co.uk. Neither source is a flat table, and turning them into modelling-ready data was a substantial part of the work. This section documents the competition selection, the relational integration, the identity resolution between the two sources, and the data-quality checks that were run against the result.

2.1 Competition selection

The selection was made by audit rather than by intuition. A script enumerates every competition-season pair available in the open-data release together with its match count, team count, date range, gender and 360-data coverage, and records an explicit decision for each. The audit covered 80 competition-season entries, of which 76 were excluded. Table 1 lists the four seasons that were kept and Table 2 summarizes why the rest were dropped.

Table 1. Competition-seasons retained for modelling. All four are complete men's domestic league seasons from 2015/16 with full event coverage and a matching Football-Data.co.uk odds file.

Competition	Season	Country	Matches	Teams	First match	Last match
La Liga	2015/2016	Spain	380	20	2015-08-21	2016-05-15
Premier League	2015/2016	England	380	20	2015-08-08	2016-05-17
Serie A	2015/2016	Italy	380	20	2015-08-22	2016-05-15
Ligue 1	2015/2016	France	377	20	2015-08-07	2016-05-14

Table 2. Reasons for exclusion, aggregated over the seventy-six rejected competition-season entries.

Reason for exclusion	Entries	Matches
women's (no odds source)	13	1,310
single-club / finals	56	705
tournament (no odds source)	6	314
league not in odds source	1	115

Why this audit was run. The open data mix full men's league seasons with tournaments, women's competitions, and season entries that contain only a single club's matches or only a cup final. Training on that mixture naively would place matches with very different scoring rates, very different amounts of prior history, and no available odds into a single training set.

What was expected. That only a small number of competition-seasons would satisfy both conditions, complete event coverage and a matching odds file.

What was observed. Exactly four did. The single largest excluded group is the set of partial season entries in which one club dominates the fixture list — the La Liga entries from 2004/05 through 2020/21, for instance, mostly contain only Barcelona or Real Madrid matches, giving a top-team share of 1.0.

What this means. The retained dataset is homogeneous in competition type, which makes rolling team form comparable across every row, at the cost of covering a single season. That trade is revisited as the principal external-validity limitation in Section 9.

2.2 Relational integration

The raw tree is parsed into five stores keyed on the identifiers that StatsBomb itself uses. The competition file supplies competition and season identifiers; the per-season match files supply the match identifier, teams, date, kick-off time and final score; the per-match event files supply the ordered event stream; and the per-match lineup files supply both squads. Every downstream join uses these identifiers and never a spelling. Labels are derived once, in a dedicated label store, from the final score in the match file, and are never recomputed downstream or taken from an odds row.

Player and team identities are resolved into canonical maps so that name variants across files collapse onto a single entity. The team map resolves to eighty canonical clubs across the four leagues and the player map to 2,547 individuals. The resolution is required to be bijective in the direction that matters: one canonical entity per source name, and no two source names silently collapsing onto the same entity.

2.3 Odds integration and identity resolution

Football-Data.co.uk publishes one CSV per league-season, with each row carrying full-time and half-time results, match statistics and pre-match bookmaker odds. There is no shared key with StatsBomb, so rows must be tagged to a match identifier by matching on pre-match identity alone. The join uses competition, date, home team and away team, and nothing else. This restriction is a leakage control, not a convenience: a Football-Data row also contains the final score and match statistics, and joining on anything derived from those fields would let the outcome enter the feature table through the back door, while also biasing the tagged sample towards fixtures where the two sources happen to agree.

Team names differ systematically between the two sources — "Ath Bilbao" against "Athletic Club", "Paris SG" against "Paris Saint-Germain" — so an alias map is built first and applied before any join. The map contains 80 entries resolved by three methods, shown in Table 3; representative non-trivial entries appear in Table 4.

Table 3. Alias-map resolution methods across the eighty club-name pairs.

Method	Entries
exact	47
fuzzy	32
override	1

Table 4. Representative alias-map entries requiring fuzzy matching or a manual override.

League	Football-Data name	StatsBomb name	Method
La Liga	Ath Bilbao	Athletic Club	fuzzy
La Liga	Ath Madrid	Atlético Madrid	override
La Liga	Betis	Real Betis	fuzzy
La Liga	Celta	Celta Vigo	fuzzy
La Liga	Espanol	Espanyol	fuzzy
La Liga	La Coruna	RC Deportivo La Coruña	fuzzy
La Liga	Levante	Levante UD	fuzzy
La Liga	Sociedad	Real Sociedad	fuzzy
La Liga	Sp Gijon	Sporting Gijón	fuzzy
La Liga	Vallecano	Rayo Vallecano	fuzzy

Table 5 reports the per-league tagging outcome over all 1,517 matches. Coverage is essentially complete. Table 6 lists every failure individually, as required, rather than summarizing them.

Table 5. Odds tagging coverage by league over the full dataset. The date-mismatch column counts rows matched on teams but requiring a date tolerance; it is zero throughout.

League	Football-Data rows	Tagged to a match	With Bet365 odds	Coverage	Date mismatches
La Liga	380	380	380	1.0000	0
Ligue 1	380	377	376	0.9895	0
Premier League	380	380	380	1.0000	0
Serie A	380	380	380	1.0000	0

Table 6. Every match excluded from the odds-tagged set, with its reason. Three fixtures are absent from the open-data release entirely; one was matched but carries no Bet365 price.

League	Date	Home	Away	Reason
Ligue 1	2015-11-22	Bastia	Ajaccio GFCO	no StatsBomb match for this fixture (absent from open-data release)
Ligue 1	2016-01-31	St Etienne	Paris SG	no StatsBomb match for this fixture (absent from open-data release)
Ligue 1	2016-04-30	Troyes	Bordeaux	no StatsBomb match for this fixture (absent from open-data release)
Ligue 1	2016-04-24	Marseille	Nantes	matched but missing Bet365 odds

What was expected. Near-complete coverage, since all four leagues are major competitions that every bookmaker prices.

What was observed. Three of the four failures are StatsBomb gaps rather than odds gaps: the fixture exists in the Football-Data file but has no counterpart in the open-data release. Only one match, Marseille against Nantes on 30 April 2016, was successfully tagged but carries no Bet365 price.

Why this matters. If coverage were partial and non-random, the market comparison in Section 6.4 would be computed on an easier or harder subset than the full test set, and the comparison would be biased. At the observed coverage the risk is negligible, and the untagged match is excluded from both sides of the comparison rather than from one.

2.4 Data-quality log

Cleaning is auditable rather than silent. Every check writes the number of records it affected to a log, and records are dropped only when a mandatory field is missing. Table 7 aggregates that log.

Table 7. Aggregated data-quality log over the cleaned store. Optional event attributes are retained as missing values rather than dropped, and are handled by the median imputer inside the modelling pipeline.

Scope	Field	Records affected	Reason
events	card	5,314,364	optional attribute absent; retained as missing
events	shot_xg	5,283,571	optional attribute absent; retained as missing
events	period,timestamp,index	439,505	events reordered into canonical event-time order
events	x	37,460	optional attribute absent; retained as missing
events	y	37,460	optional attribute absent; retained as missing

events	player_id	20,393	optional attribute absent; retained as missing
lineups	primary_position	15,569	optional attribute absent; retained as missing
lineups	country	2	optional attribute absent; retained as missing
events	team_id	0	optional attribute absent; retained as missing
lineups	jersey_number	0	optional attribute absent; retained as missing

Two entries deserve comment. The `shot_xg` and `card` fields are absent on the overwhelming majority of events simply because most events are not shots and most are not bookings; their absence is structural rather than a defect, and the aggregation code treats a missing expected-goals value as a zero contribution rather than as an unknown. The reordering entry is more consequential: 439,505 events across the 1,517 matches were moved when the store was sorted into canonical event-time order. The reason is explained in Section 3.4, and it is the single most important data-quality finding in this project. Notably, the drop log is empty — no record was discarded for a missing mandatory field.

2.5 Temporal splits

Matches are sorted by date and cut into training, validation and test blocks in that order, and every snapshot inherits its parent match's assignment. Table 8 reports the result.

Table 8. Chronological split of the 1,517 matches, with outcome shares and mean signed margin in each block. The home-win share rises materially in the test block.

Split	Matches	From	To	Home win	Draw	Away win	Mean margin
train	896	2015-08-07	2016-01-31	0.4330	0.2734	0.2935	0.3549
validation	277	2016-02-01	2016-03-18	0.4332	0.2816	0.2852	0.3032
test	344	2016-03-19	2016-05-17	0.4855	0.2297	0.2849	0.4855

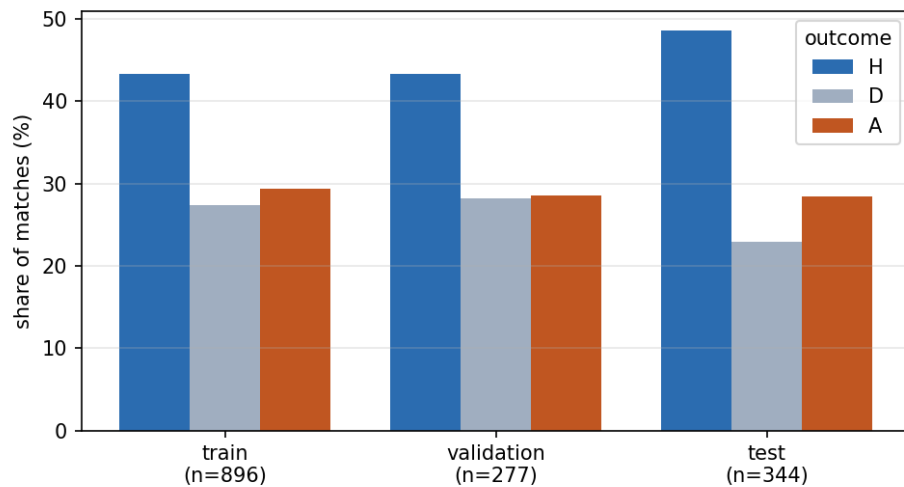


Figure 1. Outcome shares by split. The rise in home wins and fall in draws between the training and test blocks is a genuine distribution shift induced by the chronological cut, not a sampling artefact.

What was expected. Some drift, since the closing months of a season differ from the opening months in motivation, fixture congestion and squad rotation.

What was observed. The home-win rate rises from 0.4330 in training to 0.4855 in test, while the draw rate falls from 0.2734 to 0.2297, as Figure 1 shows. The mean signed margin rises correspondingly from 0.3549 to 0.4855.

Why. Late-season fixtures include a growing share of matches in which one side has something to play for and the other does not, which reduces the frequency of cagey draws and favors the stronger, usually home, team.

What it means. Every model is trained on a class balance that does not hold on the test set. This is a real cost of the chronological split, and it is one reason the models trail the market in Section 6.4. It is nevertheless the correct way to split, because a random split would let the model learn from matches played after the ones it is tested on, which is impossible at deployment and would make every reported score optimistic.

3. Feature Engineering: Pre-Match and In-Play Pipelines

Two feature tables are built. The pre-match table holds rolling prior-form aggregates for both teams and the difference between them; it has 1,517 rows and 237 columns, of which 231 enter the design matrix once identifiers, split tags and labels are removed. The in-play table holds the state of each match at each snapshot minute; it has 28,823 rows and 158 columns, and is joined to the frozen pre-match vector when the model matrices are assembled, giving 385 design columns. After one-hot encoding of the single nominal column the matrices carry 235 and 389 features respectively.

3.1 One definition per quantity

Both tables are built from a single shared module that computes twenty-nine per-team quantities from a frame of events. The pre-match builder passes a whole finished match and rolls the result into prior form; the in-play builder passes the prefix of events up to minute t . Because both go through the same function, the in-play value of a quantity and the rolling form of that same quantity are guaranteed to mean the same thing. Writing them separately would have allowed the two definitions to drift apart silently, which is precisely the class of defect that produces a model that works in development and fails in deployment.

What the quantities cover. Beyond goals and expected goals, each team's frame yields shot volume, shots on target and shots inside the box; passing volume split by pitch third and the share of passing done in the attacking third; carries and touches by third; average action position as a measure of territory; pressures applied and the share applied high up the pitch; events played under pressure; defensive actions and where they occur; fouls committed and won; turnovers; possession measured by chain ownership; and set-piece counts split into corners, free kicks and throw-ins. Each quantity is recorded both for the team and for the opponents it faced, so a team's defensive record is described as fully as its attacking one.

Possession is measured by chain, not by event count. A raw event share would credit a team for its own pressures and clearances, which by definition happen when it does not have the ball. Instead each possession chain is attributed to the team that performed the majority of the on-ball work within it, and possession share is the fraction of chains owned. The two teams' shares therefore sum to exactly one by construction, and the test suite asserts this.

Three views of every in-play quantity. Each is recorded as a match total, as a difference over the trailing ten-minute window, and as a per-minute rate. The recent window is what makes momentum visible: a team can be behind on the match totals while dominating the last ten minutes, and only the windowed view expresses that. The per-minute rate makes an early snapshot comparable with a late one, since a raw count of twelve passes at minute 10 and the same count at minute 85 describe entirely different games.

3.2 Leakage barriers

Leakage is the single largest risk in this project, because the label is derivable from the same event stream that the features come from. If a feature accidentally observes one event after the prediction time, the model can look excellent and be worthless. Four barriers are enforced, and each is a mechanical property of the code checked by an assertion or a unit test rather than a claim in prose.

Barrier one: chronological splitting

Matches are sorted by date and cut in order, as described in Section 2.5. Every snapshot inherits its parent match's split, and the builder asserts that no match has snapshots on both sides of a boundary. Cross-validation inside the training rows is grouped by match for the same reason: if snapshots of one match appeared in both the fitting and the scoring fold, the score would measure memorization of that match rather than generalization, and every model would appear far better than it is.

Barrier two: prior-match-only form

Rolling form applies a one-match lag before the window opens, so a match never contributes to its own features. Two checks enforce this. The builder asserts that each team's first appearance in the dataset carries no form at all. The test suite goes further: it rewrites a match's own scoreline to a different result and asserts that the feature row for that same match is byte-identical. This is the stronger of the two, because it would catch a lag that is correctly applied in most rows but missing in some.

Barrier three: the effective-minute prefix cut

The in-play cut must select every event at or before minute t and no event after it. This is harder than it first appears. Roughly twenty-six events across twenty-four matches carry corrupted 00:00:00 timestamps while their index positions remain correct, so sorting by timestamp would relocate those events to the start of their period and silently corrupt the prefix. The builder therefore sorts by period and index, never by timestamp, and repairs the minute column with a running maximum before cutting:

$$\text{eff}[i] = \max_{j \leq i} \text{minute}[j] \quad (3)$$

Because a running maximum never decreases, the condition that the repaired value at position i is at most t implies that the true minute of event i is also at most t . The repaired column can therefore only make the cut more conservative, never less. The cut is then guarded on both sides: the last included event must be at or before t , and the first excluded event must be strictly after t . The first assertion rules out leakage; the second rules out silently truncating the prefix and thereby weakening the model for a reason that would be invisible in the metrics.

One side effect is documented rather than hidden. The second half restarts at minute 45, so applying a running maximum across the period boundary means the opening events of the second half are not visible until the snapshot at minute 50. This makes the minute-45 snapshot slightly more conservative than it strictly needs to be. That is the correct direction in which to err, and it is why the minute-45 point on the in-play curves in Section 8B sits marginally above the trend.

Barrier four: train-only transform fitting

The median imputer, the standard scaler, the one-hot encoder and every resampler are fitted on training rows only and then applied unchanged to validation and test. Fitting a scaler on the whole table would leak the test distribution into training.

Resampling is a training-time toggle that is never written into a feature file, which matters because a resampled row is synthetic and must never be scored as though it were a real match. The pipeline raises an exception if any resampling arm is requested on the in-play snapshot table at all, for reasons given in Section 8C.

3.3 A defect found by cross-checking rather than by inspection

The first run of the aggregate builder reproduced the correct scoreline for only 1,391 of the 1,517 matches. The disagreements were exactly symmetric: one goal on the wrong side, never a wrong total. Inspecting the event stream showed why. StatsBomb records an own goal twice, as "Own Goal For" on the team that benefits and "Own Goal Against" on the team that concedes, so the credit is already on the correct side. The first implementation added the opponent's "Own Goal For" events as well, which moved exactly one goal across in every match containing an own goal. After the correction all 1,517 matches agree. The check is retained in the builder as a permanent guard, because a feature that silently disagrees with the label store is precisely the kind of error that aggregate metrics cannot reveal.

3.4 Known limitations of the feature pipelines

- *The rolling window uses a minimum period of one.* A team's second match of the season therefore carries a one-match average that is labelled as a five-match rolling form. This affects roughly the first four fixtures of every team. Discarding those matches would remove a substantial part of a training block drawn from a single season, so the artefact is recorded here rather than removed.
- *Two quantities named in the brief are implemented but not wired into the design matrices.* Pass completion rate and carries into the final third are both computed correctly by `optional_quantities()` in the shared aggregates module: completion is the share of a team's passes carrying no failure outcome, and `carries_into_attacking_third` counts carries that begin before $x = 80$ and end at or beyond it, which is entry into the third rather than activity within it. Neither feature builder calls that function, however, so neither quantity reaches the pre-match or in-play tables and no model in this report uses them. This is a wiring gap rather than a computational one, and it is reported as such rather than approximated by a proxy. Note that the distinct quantity `carries_attacking_third`, which *is* in the design matrices, counts carries whose start already lies beyond $x = 80$ and therefore measures a different thing.
- *360 freeze-frame data are unused.* The four retained seasons have zero per cent 360 coverage, as the audit in Section 2.1 records, so no imputation strategy for missing 360 frames was needed and none is claimed.

4. Paper Reproductions

Two published papers were reimplemented from scratch rather than called from a library. P1 is a data paper addressing how the data are sampled; P2 is a model paper introducing a non-deep predictive method applicable to both classification and regression. Both were checked against the hard filters before implementation began, and both carry a test suite that verifies the implementation against the equations and pseudocode as published rather than against a reference library.

Table 9. Hard-filter audit for both selected papers. Venue ranks were verified against the CORE conference portal and Scimago respectively.

Filter	P1 — G-SMOTENC	P2 — Hierarchical Shrinkage
Publication year (2022 or later)	2023 (published 28 July 2023)	2022 (ICML, 17–23 July 2022)
Venue quality	Expert Systems with Applications, Q1 (Scimago)	ICML, CORE A*
Applicable to this dataset and tasks	Yes — the pre-match table is mixed nominal and continuous, and the draw class is the minority	Yes — serves Tasks C, R and L from one method
Not a deep neural network	Oversampler, no learned model	Post-hoc regularisation of CART ensembles
Reimplementable from the paper	Algorithms 1–3 fully specified in pseudocode	Single closed-form update rule

4.1 P1 — Geometric SMOTE for nominal and continuous features

Citation. Fonseca, J. and Bacao, F. (2023). Geometric SMOTE for imbalanced datasets with nominal and continuous features. *Expert Systems with Applications*, 234, 121053.

Why this paper for this dataset. The draw outcome is the minority class, at roughly 26.5 per cent of matches overall and only 22.97 per cent of the test block. A classifier trained on that balance tends to predict draws rarely, which shows up as low recall on the draw class even when the aggregate score looks acceptable. Standard SMOTE addresses this by interpolating between a minority point and one of its neighbors along a straight line segment, but it assumes every feature is continuous. The pre-match table contains a nominal column, the competition identifier, which SMOTE can only handle by one-hot encoding it and then producing fractional values corresponding to no real category. G-SMOTENC is designed for exactly this situation.

What the method does. It replaces the linear interpolation of SMOTE with generation inside a geometric region — a hyper-spheroid centered on the minority observation and shaped by a truncation factor and a deformation factor — and it generates nominal values by taking the mode of that feature among the selected neighbors rather than interpolating. Nominal features are one-hot encoded for the distance computation, but with the non-zero constant set to the median of the standard deviations of the continuous features in the minority class divided by two, so that nominal columns carry less weight in the neighbor search than they would under plain SMOTENC.

Table 10 records the fidelity of the reimplementation against each component of the published algorithms, and names the test that verifies it.

Table 10. P1 fidelity table. Each row maps a published component of G-SMOTENC to its implementation and to the test that verifies it. All eight tests pass.

Published component	Implementation	Verification
Algorithm 1 — resample until the minority count equals the majority count	<code>'fit_resample'</code> loops per minority label, generating $ C_{maj} - C_{min} $ points	<code>'test_balances_binary'</code> , <code>'test_multiclass_balances_all_minorities'</code>
Algorithm 2, CatEncoder — one-hot with constant $\sigma_{med} / 2$	<code>'JointOneHot'</code> fitted on the combined minority and majority rows; weight = median of minority continuous standard deviations, halved	<code>'test_sigma_med_downweights_nominal'</code>
Algorithm 2, Surface — three selection strategies	<code>'minority'</code> , <code>'majority'</code> and <code>'combined'</code> implemented; <code>'combined'</code> selects x_{nn} as the nearer of the minority neighbour and the nearest majority point	exercised by all generation tests; <code>'combined'</code> is the configured default
Algorithm 3, Hyperball — uniform draw inside the unit p-ball	<code>'hyperball'</code> draws a Gaussian direction, normalises it, and scales by $r^{(1/p)}$ with r uniform	<code>'test_synthetic_continuous_bounded'</code>
Algorithm 3, Vectors — parallel and perpendicular decomposition	<code>'geometric_continuous'</code> projects onto the unit vector from centre to neighbour	<code>'test_deformation_one_collapses_to_segment'</code>
Algorithm 3, Truncate — reflect when $ \alpha_{trunc} - x_{ } > 1$	implemented verbatim as a subtraction of twice the parallel component	<code>'test_deformation_one_collapses_to_segment'</code>
Algorithm 3, Deform — subtract α_{def} times the perpendicular component	implemented verbatim; $\alpha_{def} = 1$ collapses the region to the line segment	<code>'test_deformation_one_collapses_to_segment'</code>
Algorithm 3, Translate — scale by $ x_{nn} - x_{cl} $ and shift to the centre	implemented verbatim	<code>'test_synthetic_continuous_bounded'</code>
Algorithm 3, GenNominal — mode of each nominal feature over X_{nn}	<code>'Counter(...).most_common(1)'</code> over the selected neighbour set	<code>'test_synthetic_nominal_values_are_valid'</code>
Original observations preserved	resampled matrix is the original stacked above the synthetic rows	<code>'test_no_original_rows_lost'</code>
Deterministic under a fixed seed	single <code>'numpy.random.default_rng'</code> instance threaded through generation	<code>'test_reproducible_with_random_state'</code>

Deviations from the paper, stated explicitly:

- First, Algorithm 1 as published is stated for a binary target; the reimplementation generalizes it by looping over every non-majority label independently against the shared majority pool, which is the natural extension and is required because this task has three classes.
- Second, the paper's GenNominal breaks ties in the mode by random selection, whereas the implementation uses `Counter.most_common`, which breaks ties by first-encountered order; this is deterministic rather than random but is otherwise equivalent in distribution over repeated runs.
- Third, the truncation and deformation factors are fixed at 1.0 and 0.5 rather than tuned, because tuning them would have required a hyperparameter budget that the equal-budget protocol in Section 5.3 assigns to the learners, not to the resampler.

How P1 is used. It is a training-time toggle only. It is fitted inside training rows exclusively, never written into a feature file, and never applied to the in-play snapshot table. Its evaluation is reported in Section 8C.

4.2 P2 — Hierarchical Shrinkage

Citation. Agarwal, A., Tan, Y. S., Ronen, O., Singh, C. and Yu, B. (2022). Hierarchical Shrinkage: Improving the accuracy and interpretability of tree-based models. *Proceedings of the 39th International Conference on Machine Learning*, PMLR 162:111–135.

Why this paper. It was selected over Fast Interpretable Greedy-Tree Sums, the other candidate that cleared every filter, because its core is a single closed-form post-hoc transformation with no iterative optimizer and no unstated engineering constants, which makes the Appendix A hand derivation clean and the reimplemention checkable line by line. It applies unchanged to classification and regression, so one method serves Models 1, 2 and 3 as the brief requires.

What the method does. It leaves the fitted tree structure entirely untouched and rewrites only the value stored at each node. The prediction at a leaf is re-expressed as the root mean plus one increment per split along the root-to-leaf path, and each increment is damped in proportion to how little data supported the split that produced it. A single parameter λ controls the damping. The full derivation is given in Appendix A.

Table 11 records the fidelity of the reimplemention.

Table 11. P2 fidelity table. Each row maps a published property of hierarchical shrinkage to its implementation and to the test that verifies it. All nine tests pass.

Published property	Implementation	Verification
Shrunk value equals the damped telescoping sum along the root-to-leaf path	<code>`_shrink_tree`</code> computes it as a top-down recursion off the parent	<code>`test_shrinkage_matches_the_paper_equation_on_every_path`</code> — checks every path of a fitted tree, not one example
Damping uses the parent's sample count $N(t_{l-1})$	<code>`weight = 1 / (1 + lam / counts[parent])`</code>	same test; using the child's count fails it
$\lambda \rightarrow 0$ recovers the unshrunk tree exactly	damping factor $\rightarrow 1$	<code>`test_lambda_zero_is_the_unshrunk_tree`</code>
$\lambda \rightarrow \infty$ collapses to the root mean	damping factor $\rightarrow 0$	<code>`test_large_lambda_collapses_to_the_root_mean`</code>
Shrinkage is monotone towards the root in λ	consequence of the damping form	<code>`test_shrinkage_is_monotone_towards_the_root`</code>
Classification values remain on the probability simplex	convex combination of simplex points; only floating-point dust is clipped	<code>`test_classifier_output_is_a_valid_probability_vector`</code>
Tree structure is unchanged	only <code>`tree.value`</code> is rewritten; splits and thresholds are untouched	<code>`test_own_leaf_descent_matches_sklearn_apply`</code>
Applies to a single tree and to an ensemble	<code>`HSTree*`</code> and <code>`HSForest*`</code> classes for both classification and regression	<code>`test_forest_variants_fit_predict_and_beat_a_constant`</code>
λ selected by cross-validation	<code>`_select_lambda`</code> over the grid $\{0, 0.1, 1, 10, 25, 50, 100\}$ inside training rows only	<code>`test_cv_selects_a_lambda_from_the_grid_without_touching_validation`</code>

Deviations from the paper, stated explicitly.:

- First, the base learner is a scikit-learn random forest rather than a from-scratch CART implementation; the reimplemented contribution is the shrinkage transformation, which is what the paper introduces, and the paper itself is explicit that the method is applied on top of a standard fitted ensemble.
- Second, λ is selected by minimizing log loss for classification and squared error for regression on internal folds, whereas the paper reports results across a grid; selecting one value was necessary to place the method on an equal footing with the tuned library baselines under the protocol in Section 5.3.

An adaptation required by this project. The paper demonstrates classification and regression but does not address explanation. Explaining the P2 model with TreeSHAP required a fix that is worth stating, because without it the explanations would have been quietly wrong. The implementation stores shrunk node values in a separate array alongside the fitted forest rather than inside the tree structure. TreeSHAP reads the tree structure directly, so an explainer pointed at the fitted forest would have explained the unshrunk forest, produced entirely plausible-looking attributions, and raised no error. The `shap_base_estimator` method now writes the shrunk values back into the tree before explanation, and a unit test asserts that the explained estimator reproduces the shrunk model's predictions exactly. This is a good example of a failure that is invisible in the output and detectable only by checking the mechanism.

5. Experimental Setup

5.1 Splits and snapshotting

The chronological split described in Section 2.5 gives 896 training, 277 validation and 344 test matches. Snapshotting at nineteen regulation minutes per match expands these to 17,024 training and 6,536 test snapshots for Tasks Lc and Lr. The three splits have three distinct and non-overlapping roles, and each is used for exactly one purpose:

- **Training** fits every model, and hosts the internal cross-validation used for hyperparameter search and for the P2 λ selection.
- **Validation** fits the probability calibrators, fits the two margin-to-probability links of Section 6.5, and hosts every ablation in Section 8. No test row is read during any of these.
- **Test** is read exactly once, for the final comparisons reported in Sections 6, 7 and 8. It is never used for model selection of any kind.

5.2 The model zoo

Table 12 lists every learner, the tasks it is applied to, and the role it plays in the comparison. The constant baseline is not decoration: in a domain where the best available forecaster is only modestly better than the base rate, a model that cannot clearly beat the prior has not demonstrated anything, and Section 6.6 shows that this is exactly what happens on the pre-match task.

Table 12. The model zoo. Kernel methods are restricted to the pre-match tasks for the reason measured in Section 8D.

Model	Library	Tasks	Role in the comparison
Dummy prior	<code>`sklearn.dummy`</code>	C, R, L	Sanity floor. Predicts the training class prior for C and Lc and the mean training margin for R and Lr. For Task L the frozen pre-match prediction is used as a second baseline.
Random forest	<code>`sklearn.ensemble`</code>	C, R, L	Low-tuning bagged ensemble; the variance-reduction reference.
Gradient boosting (GBM)	<code>`sklearn.ensemble`</code>	C, R, L	Textbook boosting, via the histogram-based implementation.
XGBoost	<code>`xgboost`</code>	C, R, L	Regularised tree boosting; the usual state of the art on tabular data.
LightGBM	<code>`lightgbm`</code>	C, R, L	Histogram-based boosting; compared against XGBoost and GBM on the large snapshot table.
Kernel SVM / SVR	<code>`sklearn.svm`</code>	C, R	First kernel method. RBF kernel; scale-sensitive and drives the $O(n^2)$ discussion.
Kernel ridge, exact	<code>`sklearn.kernel_ridge`</code>	R	Second kernel method, solved exactly.
Kernel ridge, Nyström	<code>`sklearn.kernel_approximation`</code>	R	Approximate kernel; compared directly against the exact solve.
P2 hierarchical shrinkage	reimplemented from scratch	C, R, L	The P2 paper method, serving all three deliverables.
De-vigged market odds	derived from tagged odds	C	The main external baseline. Every Task C model is reported against it.

Kernel methods are restricted to the pre-match tasks. This is not an arbitrary exclusion. An exact kernel method requires an n -by- n matrix of pairwise similarities, and at 17,024 training snapshots that matrix alone would occupy more than two gigabytes before any solve is attempted. Section 8D measures this rather than assuming it, and the Nyström variant is retained in the pre-match zoo

specifically so that its accuracy cost can be seen next to the exact method on a problem small enough for both.

5.3 Hyperparameter search protocol

Comparing models is fair only if each was given the same opportunity to be configured well. The search therefore assigns every model within a task an identical candidate budget of twelve configurations drawn from a random search over a per-model grid, and the equality is asserted at the end of the run rather than merely described. Scoring is by three-fold cross-validation inside the training rows only, so neither the validation nor the test split influences the choice. On the snapshot table the folds are grouped by match, for the reason given in Section 3.2. The classification objective is log loss and the regression objective is mean absolute error, matching the quantities the report cares about rather than arg-max accuracy.

Table 13 reports the outcome. The gap between the best and the median candidate is the informative column: it shows how much the search actually moved each learner. A small gap means the model was already close to its best configuration and that tuning was not the limiting factor.

Table 13. Hyperparameter search. Equal candidate counts within every task confirm the equal-budget protocol. Scores are cross-validated inside the training rows; lower is better for both objectives.

Task	Model	Candidates	Folds	Tuning rows	Objective	Best	Median
C	gbm	12	3	896	log_loss	1.15578	1.60849
C	kernel_svm	12	3	896	log_loss	1.04565	1.06703
C	lightgbm	12	3	896	log_loss	1.11193	1.30327
C	p2_hier_shrinkage	12	3	896	log_loss	1.02980	1.03323
C	random_forest	12	3	896	log_loss	1.02736	1.03575
C	xgboost	12	3	896	log_loss	1.06899	1.22903
R	gbm	12	3	896	mae	1.25590	1.29476
R	kernel_ridge_exact	12	3	896	mae	1.26156	1.29695
R	kernel_ridge_nystroem	12	3	896	mae	1.25913	1.29571
R	kernel_svr	12	3	896	mae	1.26908	1.32609
R	lightgbm	12	3	896	mae	1.24136	1.28316
R	p2_hier_shrinkage	12	3	896	mae	1.22909	1.24213
R	random_forest	12	3	896	mae	1.23222	1.23435
R	xgboost	12	3	896	mae	1.24223	1.29756
Lc	gbm	12	3	7,999	log_loss	1.13881	1.53980
Lc	lightgbm	12	3	7,999	log_loss	0.98075	1.22720
Lc	p2_hier_shrinkage	12	3	7,999	log_loss	0.86832	0.89224
Lc	random_forest	12	3	7,999	log_loss	0.81227	0.85174
Lc	xgboost	12	3	7,999	log_loss	0.86490	1.19215
Lr	gbm	12	3	7,999	mae	0.96849	1.01012
Lr	lightgbm	12	3	7,999	mae	0.96492	0.98800
Lr	p2_hier_shrinkage	12	3	7,999	mae	0.93218	0.95283
Lr	random_forest	12	3	7,999	mae	0.90722	0.92397
Lr	xgboost	12	3	7,999	mae	0.93843	0.99511

What was expected. That the boosted models would move most under search, because they have the most parameters that materially change their behavior, and that the random forest would move least.

What was observed. Exactly that pattern. On Task C the gap between best and median is 0.45271 for GBM and 0.16004 for XGBoost, against 0.00839 for the random forest and 0.00343 for the P2 model.

Why. An unrestricted boosted ensemble with a high learning rate overfits 896 training rows badly, so the worst candidates in its grid are very poor and the median is dragged upwards. A bagged forest averages over trees and is insensitive to most of its grid.

What it means. The tuning budget was adequate: the best candidates for the low-variance models were found early and further budget would not have helped them, while the boosted models genuinely needed the search to reach a sensible configuration. The selected configurations are listed in Appendix B.

5.4 Calibration strategy

A model can rank matches well and still state badly wrong confidences. For a forecasting product the stated probability *is* the output, so calibration is not a cosmetic step. Every probabilistic classifier is calibrated on the validation split with the base estimator frozen, which means the calibrator can neither refit the model nor observe test rows. Isotonic regression is used, with Platt scaling available as a fallback.

An implementation detail that affects the reported numbers. Isotonic regression fitted on a few hundred validation rows can output exact zeros, which are undefined under log loss. Probabilities are therefore floored at a small value and renormalized. The identical floor is applied to the uncalibrated probabilities as well, so that the before-and-after comparison in Section 7 isolates the effect of the calibrator rather than the effect of the floor.

5.5 Statistical testing protocol

Every results table in this report orders models by a single number computed on one test set of a few hundred matches. A difference in such numbers is not by itself evidence that one model is better than another, so two distinct forms of repetition are run, answering two distinct questions.

Seed repetition refits every model under 3 random states on the fixed splits. It measures stability — whether a model returns the same answer when only its random seed changes — and nothing else. It must not be used to claim superiority, for a specific reason that was observed directly during this project: the only quantity varying between replicates is the random state, so a deterministic model such as the constant baseline has zero spread. A paired test across seeds then divides by approximately zero and reports an extremely small p-value for any difference at all, however small that difference is relative to the noise that actually matters. In an earlier version of this analysis such a test declared the constant baseline significantly different from a boosted model, while the correct test below found no detectable difference between any pair on that task. The seed columns are therefore reported as a stability diagnostic only, and the code names the corresponding flag `gap_exceeds_seed_noise` rather than significant.

The match-clustered bootstrap is the only test in this report that licenses a claim that one model is better than another. It resamples the quantity that actually limits the conclusion: the finite set of test matches. Matches are resampled rather than rows, because the nineteen

snapshots of one match are strongly dependent; a row-level bootstrap would treat them as independent and understate the variance by roughly the cluster size, producing confidence intervals that are far too narrow and p-values that are far too small. Holm–Bonferroni correction is applied within each task, because comparing many pairs raises the chance that at least one appears significant by accident.

6. Results

This section reports the complete metric suite for every model on every task, including where performance is poor. No model is omitted and no metric is selected after the fact. All figures are computed on the test split, which was read once.

6.1 Model 1 — Pre-match outcome classification

Table 14 reports the probabilistic metrics and Table 15 the per-class breakdown. The test block contains 167 home wins, 79 draws and 98 away wins.

Table 14. Task C probabilistic results on the test split, ranked by ranked probability score. Expected calibration error is shown before and after isotonic calibration. Lower is better throughout.

Model	RPS	Log loss	Brier	ECE before	ECE after
dummy	0.22812	1.05455	0.63585	0.05243	0.05225
random_forest	0.21962	1.16692	0.63295	0.04257	0.08248
gbm	0.21862	1.08193	0.62132	0.12474	0.06405
kernel_svm	0.22482	1.12305	0.63323	0.05589	0.01438
p2_hier_shrinkage	0.21903	1.12813	0.62724	0.08685	0.08730
xgboost	0.22108	1.10784	0.63729	0.06684	0.08070
lightgbm	0.21891	1.06616	0.62195	0.11808	0.06694

Table 15. Task C per-class precision, recall and F1 on the test split. The draw column is the one that matters for the imbalance study in Section 8C.

Model	P (H)	R (H)	F1 (H)	P (D)	R (D)	F1 (D)	P (A)	R (A)	F1 (A)
dummy	0.4855	1.0000	0.6536	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
random_forest	0.5636	0.7425	0.6408	0.2373	0.1772	0.2029	0.5231	0.3469	0.4172
gbm	0.5561	0.7425	0.6359	0.2258	0.0886	0.1273	0.4667	0.4286	0.4468
kernel_svm	0.5502	0.6886	0.6117	0.1538	0.0253	0.0435	0.3934	0.4898	0.4364
p2_hier_shrinkage	0.5631	0.7485	0.6427	0.2456	0.1772	0.2059	0.5077	0.3367	0.4049
xgboost	0.5907	0.6826	0.6333	0.2381	0.1899	0.2113	0.4318	0.3878	0.4086
lightgbm	0.5667	0.7126	0.6313	0.1935	0.0760	0.1091	0.3981	0.4184	0.4080

What was expected. That the boosted ensembles would lead on the probabilistic score, that all learners would sit close together, and that every model would struggle badly on the draw class.

What was observed. The best ranked probability score is 0.21862 from GBM, against 0.22812 for the constant baseline — a difference of 0.0095, or four per cent of the baseline value. The entire spread across seven models, from 0.21862 to 0.22812, is smaller than the gap to the market baseline reported in Section 6.4. On the draw class every model is poor: the best draw recall is 0.1772 from the random forest and the best draw F1 is 0.2113 from XGBoost, while the constant baseline never predicts a draw at all and scores zero on all three draw columns.

Why. Two mechanisms combine. First, pre-match features describe recent form, and recent form is a weak predictor of a single match outcome; the achievable ceiling is low regardless of the learner. Second, the draw is intrinsically hard to predict because it is not a distinct kind of match but the boundary between two others — a draw is what happens when two roughly equal teams meet and neither converts. There is no region of feature space where draws dominate, so no classifier maximizing expected accuracy will ever place a draw at the arg-max for many matches.

What the metrics reveal. The near-agreement of every learner is more informative than their ordering. When a constant baseline and a tuned gradient-boosted ensemble land within 0.01 RPS of each other, the features rather than the model capacity are setting the ceiling. Section 6.6 confirms formally that none of these differences survives statistical testing.

Comparison against expectation. The expectation that boosted models would lead was met, but only nominally; the expectation that the models would clearly beat the prior was not.

6.2 Model 2 — Pre-match goal-margin regression

Table 16 reports the regression results. The constant baseline predicts the mean training margin for every match.

Table 16. Task R results on the test split, ranked by mean absolute error. The correlation column separates the ability to rank matches by expected margin from the ability to state the margin on the right scale.

Model	Training rows	MAE	RMSE	Correlation
dummy	896	1.48997	1.89230	0.00000
random_forest	896	1.36653	1.71419	0.42091
gbm	896	1.39335	1.75006	0.37678
kernel_svr	896	1.41018	1.78076	0.35534
kernel_ridge_exact	896	1.41131	1.78156	0.35323
kernel_ridge_nystroem	896	1.41534	1.78832	0.35162
p2_hier_shrinkage	896	1.36651	1.70884	0.43067
xgboost	896	1.36891	1.72150	0.41221
lightgbm	896	1.38381	1.72750	0.40354

What was expected. That pre-match margin prediction would be only slightly better than predicting the average margin for every match, because the pre-match features describe recent form and say nothing about how the match will actually unfold.

What was observed. The best mean absolute error is 1.36651 from the P2 model, against 1.48997 for the constant baseline — an improvement of 0.123 goals, or 8.3 per cent. The random forest is within 0.00002 of the P2 model. The three kernel methods cluster tightly between 1.41018 and 1.41534, and the exact and Nyström kernel ridge solutions differ by only 0.004 in mean absolute error.

Why. The correlation column makes the mechanism visible. The best pre-match model reaches a correlation of 0.43067 with the realized margin, and the constant baseline reaches exactly zero by construction. A correlation near 0.43 means the model orders matches by

expected margin with real but modest skill. It is forecasting a quantity that is mostly determined by events that have not yet happened, so the ceiling is low for any learner.

What it means. The near-identical performance of the exact and approximate kernel ridge solutions is a useful secondary finding: the Nyström approximation with 250 landmark components loses essentially nothing in accuracy on this problem while, as Section 8D shows, costing a small fraction of the fit time.

Comparison. The gap between the best model and the baseline here (8.3 per cent) is larger than the corresponding gap on Task C (4.2 per cent), which suggests the margin target carries slightly more learnable structure than the three-way label.

6.3 Model 3 — In-play prediction

Tables 17 and 19 report the aggregate in-play results over all 6,536 test snapshots. These are aggregates over all nineteen minutes; the decomposition by minute, which is the more informative view, is given in Section 8B.

Table 17. Task Lc probabilistic results, aggregated over all nineteen snapshot minutes of the test matches.

Model	RPS	Log loss	Brier	ECE before	ECE after
dummy	0.22812	1.05455	0.63585	0.05243	0.05225
random_forest	0.14673	0.78838	0.45510	0.02089	0.04638
gbm	0.15736	0.84365	0.48813	0.15670	0.04793
p2_hier_shrinkage	0.15010	0.81069	0.46665	0.09885	0.04930
xgboost	0.15016	0.79612	0.46376	0.03683	0.05916
lightgbm	0.15366	0.82962	0.47648	0.06045	0.04315

Table 18. Task Lc per-class precision, recall and F1, aggregated over all snapshot minutes.

Model	P (H)	R (H)	F1 (H)	P (D)	R (D)	F1 (D)	P (A)	R (A)	F1 (A)
dummy	0.4855	1.0000	0.6536	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
random_forest	0.7419	0.7888	0.7646	0.4361	0.4137	0.4246	0.6697	0.6251	0.6467
gbm	0.6998	0.8090	0.7505	0.3962	0.2951	0.3383	0.6257	0.5881	0.6063
p2_hier_shrinkage	0.7455	0.7765	0.7607	0.3998	0.4610	0.4282	0.6953	0.5602	0.6205
xgboost	0.7273	0.7992	0.7616	0.4081	0.3504	0.3771	0.6443	0.6090	0.6262
lightgbm	0.7199	0.8172	0.7655	0.4039	0.3418	0.3703	0.6304	0.5634	0.5950

Table 19. Task Lr results, aggregated over all snapshot minutes.

Model	Training rows	MAE	RMSE	Correlation
dummy	17,024	1.48997	1.89230	0.00000
random_forest	17,024	0.97087	1.28670	0.73310
gbm	17,024	1.02270	1.34480	0.70265
p2_hier_shrinkage	17,024	0.98092	1.30981	0.72015
xgboost	17,024	1.00771	1.32356	0.72070
lightgbm	17,024	1.02268	1.34111	0.71187

What was expected. A large improvement over the pre-match tasks, because after an hour of play the current score is already most of the answer.

What was observed. The best in-play ranked probability score is 0.14673 against 0.21862 pre-match, a reduction of 33 per cent. The best in-play mean absolute error is 0.97087 against 1.36651 pre-match, a reduction of 29 per cent. Draw recall rises from 0.1772 pre-match to 0.4137 in-play for the random forest, and the margin correlation rises from 0.43067 to 0.73310.

Why. The dominant in-play feature is the current goal difference, and its predictive power grows as the time remaining shrinks. The draw-recall improvement has the same cause and is worth stating separately: a goalless match at minute 80 is a genuinely distinct state that the model can recognize, whereas pre-match there is no such state. This is the clearest demonstration in the report that the draw class is hard pre-match because of missing information rather than because of class imbalance.

What it means. The gain from moving to the in-play setting is far larger than the gain from any choice of learner within a setting. That is the central practical finding of this project and it recurs in every subsequent section.

A caveat on the constant baseline row. The dummy learner's Lc row is numerically identical to its C row, at 0.22812 RPS. This is correct rather than an error: the dummy predicts the training class prior and ignores its input entirely, so replicating each match nineteen times cannot change its score.

6.4 Comparison against the de-vigged market

A metric on its own does not say whether a forecast is good. The betting market provides a demanding external reference: published odds represent the aggregated opinion of participants with money at stake. Bookmaker decimal odds embed a margin, so the three implied probabilities sum to more than one; they are de-vigged by taking the reciprocal of each decimal odd and normalizing the three to sum to exactly one.

Test-set discipline. The comparison is restricted to test matches that carry odds, and every model is re-scored on exactly that subset, so both sides are evaluated on identical matches. Table 20 reports the coverage of that subset by competition; Table 21 is the comparison itself.

Table 20. Odds tagging coverage of the test split by competition. The single untagged match is excluded from both sides of the comparison.

Competition	Test matches	Tagged with odds	Coverage
La Liga	89	89	1.0000
Ligue 1	78	77	0.9872
Premier League	87	87	1.0000
Serie A	90	90	1.0000
ALL	344	343	0.9971

Table 21. Task C models against the de-vigged market on the 343 odds-tagged test matches. A positive gap means the model is worse than the market.

Model	Matches	RPS	Log loss	Brier	ECE	RPS gap	Beats market
MARKET_devigged	343	0.19665	0.95749	0.56771	0.02719	+0.00000	no
gbm	343	0.21878	1.08034	0.62030	0.06555	+0.02213	no
lightgbm	343	0.21890	1.06401	0.62044	0.06530	+0.02225	no
p2_hier_shrinkage	343	0.21919	1.12728	0.62642	0.08601	+0.02254	no
random_forest	343	0.21986	1.16648	0.63243	0.08147	+0.02321	no
xgboost	343	0.22128	1.10697	0.63660	0.07956	+0.02463	no
kernel_svm	343	0.22513	1.12300	0.63303	0.01556	+0.02848	no
dummy	343	0.22839	1.05393	0.63542	0.05367	+0.03174	no

What was expected. That the models would not beat the market. The market observes team news, injuries, suspensions, expected line-ups and the flow of money, none of which appear in these feature tables. The purpose of the comparison is to measure the size of the gap, not to win it.

What was observed. The de-vigged market scores 0.19665. The best model scores 0.21878, a gap of +0.02213. Zero of seven models beat the market. The gap for the constant baseline is +0.03174, so the models close roughly thirty per cent of the distance from the prior to the market.

A striking detail. The market's draw recall is undefined — the de-vigged market never assigns the draw the highest probability in any of the 343 matches, and scores zero on all three draw columns exactly as the constant baseline does. It nonetheless achieves the best probabilistic score of any forecaster here by a wide margin. This is the single clearest illustration in the report that arg-max accuracy on the minority class and probabilistic quality are different objectives: the market never *predicts* a draw yet states the *probability* of a draw better than anything else in the table.

Why the models trail. Two reasons, different in kind. The first is an information gap: the market knows the starting eleven and this pipeline does not. The second is a feature gap of this project's own making, and the ablation in Section 8 quantifies it. The third contributing factor is the distribution shift documented in Section 2.5 — the models are trained on a 43.3 per cent home-win rate and tested on 48.6 per cent, while the market prices each fixture from current information and is unaffected.

What it means. The result is the expected one and is reported as a finding rather than a disappointment. Beating a liquid market on four major leagues using form features from a single season would be a surprising claim requiring far stronger evidence than this project can supply. The practical conclusion is that the market should remain in the pipeline as a benchmark, and that closing the gap is a feature-engineering problem before it is a modelling problem.

6.5 Converting the Model 2 margin into Model 1 probabilities

The brief asks whether the regressed margin can be turned into useful outcome probabilities. This is a question about whether a model trained on a richer target carries information that a model trained on the three-way label does not. Two links were fitted on the validation split and scored once on test. The ordinal link treats the realized margin as a

latent continuous quantity cut by two symmetric thresholds, which is the natural assumption for an ordered outcome:

$$P(A) = \Phi\left(\frac{-\theta - \hat{m}}{\sigma}\right), \quad P(H) = 1 - \Phi\left(\frac{\theta - \hat{m}}{\sigma}\right), \quad P(D) = 1 - P(H) - P(A) \quad (4)$$

In equation (4), $\hat{m}$ is the predicted margin, θ is the half-width of the draw band and σ is the spread of the realized margin around the prediction. Both are fitted by maximizing the likelihood on validation rows and never on test rows; the fitted values are $\theta = 0.597$ and $\sigma = 1.46979$. A multinomial logistic link on the predicted margin and its square was fitted as a second, more flexible alternative.

Table 22. Margin-to-probability conversion against the classifier trained directly for the task, on the test split. Both sides were selected on validation and scored once.

Converter	Source	RPS	Log loss	Brier	ECE
model2_ordinal[p2_hier_shrinkage]	Task R margin	0.20856	0.99964	0.59552	0.05416
model1_direct[random_forest]	Task C classifier	0.20939	1.00382	0.59937	0.04257
model2_logistic[p2_hier_shrinkage]	Task R margin	0.21029	1.01183	0.60063	0.06012

What was expected. That the converted regressor would be competitive but slightly worse, because Task C optimizes the exact quantity being scored while Task R optimizes a proxy.

What was observed. The direct classifier scores 0.20939 and the ordinal conversion 0.20856, so the conversion is marginally *better* by 0.00083. On calibration the ordering reverses: the ordinal conversion records an expected calibration error of 0.05416 against 0.04257 for the direct classifier. The more flexible logistic link is worst on both counts, at 0.21029 RPS.

Why. The ordinal link is a two-parameter function of a single predicted number. It cannot produce erratic probabilities, because it is smooth and monotone in the predicted margin by construction. The direct classifier has far more freedom and can place sharp probabilities in regions of feature space where it saw few training matches. On a small pre-match training set that freedom costs calibration. The conversion is better calibrated because it is more constrained, not because it knows more. That the logistic link — which has more freedom than the ordinal link — is worse than both supports this reading directly.

Does this support the original expectation? Only partly. The expectation that the direct classifier would rank better was not met, though the difference is tiny. The calibration reversal was not anticipated and is the more interesting result.

Limitations. Both numbers come from one test split of 344 matches and the difference in ranked probability score is 0.0008; Section 6.6 shows that differences of this size on this dataset are not separable from noise, so this should not be read as a reliable ordering.

Conclusion. The margin can be converted into useful probabilities, and the conversion is a reasonable component for a system that must produce both an expected scoreline and an outcome distribution from a single fitted model.

6.6 Are the observed differences real?

Table 23 summarizes the match-clustered bootstrap over all 87 pairwise comparisons, and Table 41 reports the seed-stability diagnostic. Table 40 lists every comparison that survives Holm correction.

Table 23. Match-clustered bootstrap results by task, after Holm–Bonferroni correction within each task.

Task	Pairwise comparisons	Significant after correction
C	21	0
R	36	6
Lc	15	6
Lr	15	9

What was observed. Of 87 comparisons, 21 survive correction. On Task C, 0 of 21 survive. On Task R, 6 of 36. On the in-play tasks the picture is completely different: 6 of 15 on Lc and 9 of 15 on Lr.

Why the pre-match and in-play tasks differ so sharply. The effect sizes differ by an order of magnitude. On the pre-match task the differences between learners are a few thousandths of a ranked probability score, against a match-to-match spread that is far larger, so 344 matches cannot resolve them. On the in-play tasks the differences between a real model and the constant baseline are large, because the in-play features genuinely carry information. The statistical machinery is identical in both cases; what changes is whether a real effect exists that is large enough to detect. Note that on Task R the surviving comparisons are all against the constant baseline — no learner is distinguishable from any other learner.

What the seed table shows and does not show. Every learner is highly reproducible; the largest standard deviation in the table is a few ten-thousandths of a metric unit, and the constant baseline is exactly deterministic with zero spread. This confirms that the pipeline is reproducible under reseeding. It says nothing whatever about which model is better, for the reason given in Section 5.5.

What this means for the rest of the report. The pre-match leaderboard in Sections 6.1 and 6.2 should not be read as a ranking. Where the bootstrap finds no difference, the honest statement is that the models are indistinguishable on this evidence, not that the one at the top is best. The in-play results are on much firmer ground.

Would more data help? Probably not usefully. The pre-match effects are so small that separating them would require far more data than one season provides, and a difference that small would not matter in any practical deployment.

7. Calibration and Reliability Analysis

7.1 Effect of calibration on the pre-match and in-play classifiers

The expected calibration error before and after isotonic calibration is reported in Tables 14 and 17. Figures 2 and 3 give the corresponding reliability diagrams, plotting observed accuracy against mean predicted confidence within each confidence bin; a perfectly calibrated model lies on the diagonal.

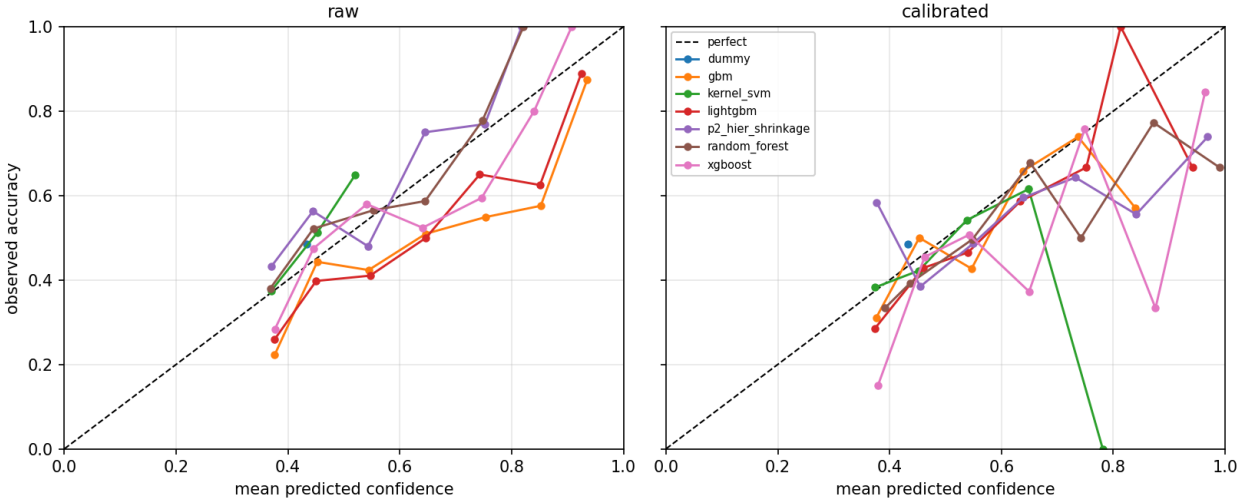


Figure 2. Task C reliability diagrams, raw (left) and after isotonic calibration (right). Points below the diagonal indicate over-confidence.

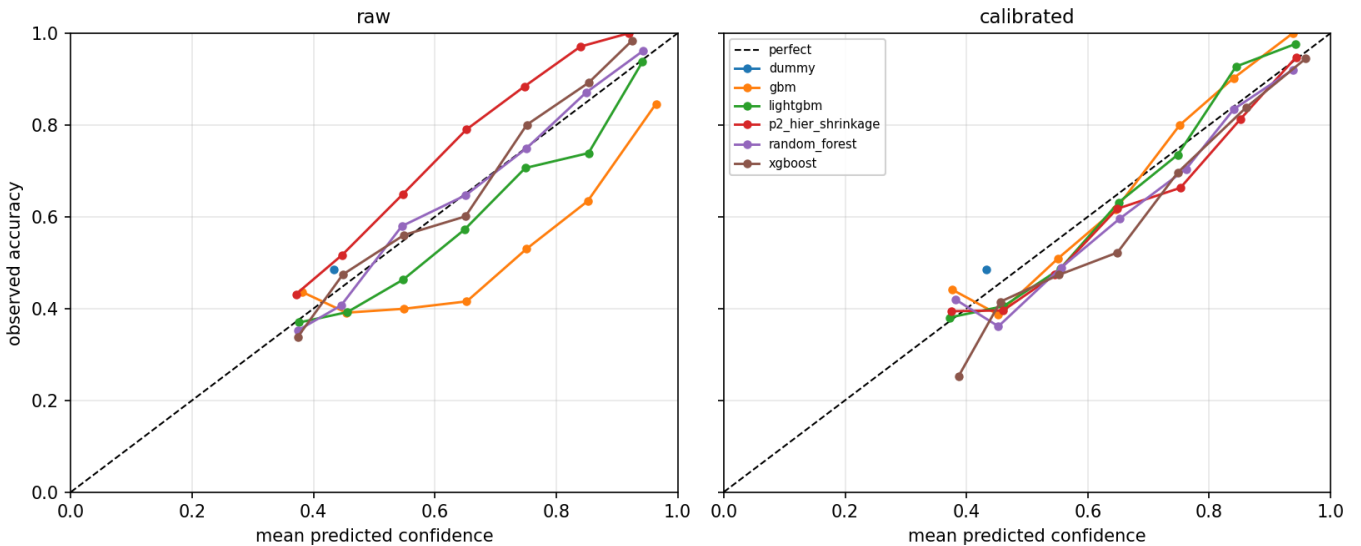


Figure 3. Task Lc reliability diagrams, raw (left) and after isotonic calibration (right). The in-play models span a far wider confidence range than the pre-match models, because late-match snapshots are genuinely near-certain.

What was expected. Boosted models to be noticeably over-confident before calibration and to improve substantially after it; the constant baseline to be almost perfectly calibrated already, since it predicts the training prior and has no confidence to misstate; and the P2 model to need less correction than the boosted models, because averaging shrunk leaf values pulls extreme estimates towards better-supported ancestors.

What was observed. The pattern matches on every count. The largest calibration gain is GBM on Task Lc, where the expected calibration error falls from 0.15670 to 0.04793, a reduction of 69 per cent. LightGBM on Task C falls from 0.11808 to 0.06694. The constant baseline barely moves, from 0.05243 to 0.05225. Against expectation in one case, calibration makes the random forest on Task C *worse*, moving from 0.04257 to 0.08248, and the P2 model on Task C is essentially unchanged at 0.08685 against 0.08730.

Why. Boosted models optimize a loss that rewards separating classes, and with unrestricted depth they push probabilities towards zero and one, which is exactly the behaviour that produces a large calibration error and a large subsequent correction. Models that already output averaged leaf proportions, such as the random forest and the shrunk forest from P2, start much closer to calibrated and have little left to gain. Where calibration slightly worsens the error, the cause is the small validation set: isotonic regression is flexible and can overfit a few hundred rows, fitting noise in the reliability curve and transferring some of that noise to the test set. This is a pipeline effect caused by the amount of calibration data available, not a defect of the underlying model, and it is listed as a limitation in Section 9.

What it means. Calibration is doing real work but it cannot manufacture information. On Task C the best ranked probability score after calibration is 0.21862, against 0.14673 on Task Lc. Calibration moves the expected calibration error by a few hundredths; the move from pre-match to in-play information moves the ranked probability score by far more.

Conclusion. Calibration should be kept, because the stated probability is the product and correcting it is cheap, but it is not where the performance of this system comes from.

7.2 Per-phase calibration for Model 3

A single calibration number averaged over all nineteen snapshot minutes would combine an easy problem and a hard one and conceal both. Table 24 therefore reports calibration by game phase, averaged over the five real learners; Table 42 gives the per-model breakdown. Over-confidence is defined as mean stated confidence minus realized accuracy, so a positive value means the model claims more certainty than it earns in that phase.

Table 24. Model 3 calibration by game phase, averaged over the five non-baseline learners. The snapshot count is per phase across all test matches.

Phase	Snapshots	ECE	Mean confidence	Accuracy	Over-confidence	RPS
0-15	1,032	0.09673	0.57490	0.48411	+0.09079	0.22123
15-30	1,032	0.10157	0.60799	0.51027	+0.09772	0.20767
30-45	1,032	0.07686	0.63624	0.56512	+0.07113	0.18311
45-60	1,032	0.06267	0.68927	0.64031	+0.04897	0.14648
60-75	1,032	0.05324	0.73512	0.71822	+0.01690	0.11543
75-90	1,376	0.08149	0.77722	0.85552	-0.07830	0.06469

What was expected. Rising confidence and rising accuracy through the match, with the two not necessarily rising at the same rate.

What was observed. Mean confidence rises steadily from 0.57490 in the opening phase to 0.77722 in the closing phase, while accuracy rises faster, from 0.48411 to 0.85552. The consequence is a sign change: models are over-confident by +0.09079 in the 0–15 minute phase and *under*-confident by -0.07830 in the 75–90 minute phase. The ranked probability score falls monotonically across phases, from 0.22123 to 0.06469.

Why. Early in a match the model has almost no live information but a calibrator fitted over all minutes has taught it to state a confidence typical of the average snapshot, which is too high for minute 5. Late in a match the opposite holds: a two-goal lead with ten minutes remaining is very nearly decisive, but the globally fitted calibrator pulls the stated probability back towards the average and understates the model's real certainty. The single global calibrator is therefore mis-specified at both ends of the match and approximately correct in the middle, which is exactly what the 45–60 phase shows with an over-confidence of only +0.048.

What the per-model view adds. The effect is uniform across learners rather than a property of one of them, which rules out an artefact of a single model's probability shape. The P2 model shows the largest mid-match over-confidence, at +0.11682 in the 15–30 phase, and the random forest the smallest closing-phase distortion at +0.10676.

Conclusion. If these probabilities were shown to a user during a live match, a phase-specific calibrator would be clearly preferable to the single global one used here. That change is straightforward — fit one isotonic regressor per phase on the validation snapshots — and is the most concrete improvement this report can recommend from its calibration analysis.

8. Analysis

8A. Error and Failure Analysis

Aggregate metrics say how much error there is but not what kind. This section asks what the models are actually using, where they fail worst, and whether those failures are mistakes that better modelling could fix or outcomes that no forecaster could have called. Attributions are computed with TreeSHAP, which is exact for tree ensembles rather than a sampled approximation. Its output is additive: the contributions for one prediction sum to the difference between that prediction and the model's average output, which makes each explanation checkable.

Global attributions

Table 25 ranks features by mean absolute SHAP value, and Figures 4 through 7 give the corresponding beeswarm plots. In a beeswarm each point is one test row, horizontal position is that feature's contribution to the model output, and colour is the feature value, so a clean color gradient from one side to the other indicates a consistent, monotone effect.

Table 25. Mean absolute SHAP value per feature, the six largest per task. This ranks influence on the model, which is not the same as causal importance in football.

Task	Model	Feature	Mean SHAP
C	random_forest	away_venue_form_xga	0.022888
C	random_forest	diff_venue_form_xga	0.017662
C	random_forest	diff_form_agg_passes	0.009298
C	random_forest	diff_form_agg_opp_shots_on_target	0.008290
C	random_forest	diff_form_agg_events	0.006967
C	random_forest	diff_form_agg_shots_on_target	0.006404
R	p2_hier_shrinkage	away_venue_form_xga	0.147635
R	p2_hier_shrinkage	diff_venue_form_xga	0.077150
R	p2_hier_shrinkage	diff_form_agg_carries	0.068644
R	p2_hier_shrinkage	home_form_agg_passes	0.053217
R	p2_hier_shrinkage	diff_form_agg_passes	0.042500
R	p2_hier_shrinkage	diff_form_agg_events	0.033124
Lc	random_forest	inplay_diff_goals	0.056485
Lc	random_forest	inplay_goal_diff	0.054075
Lc	random_forest	inplay_rate_diff_goals	0.050497
Lc	random_forest	inplay_away_goals	0.014888
Lc	random_forest	diff_venue_form_xga	0.012059
Lc	random_forest	away_venue_form_xga	0.011297
Lr	random_forest	inplay_goal_diff	0.244491
Lr	random_forest	inplay_diff_goals	0.233455
Lr	random_forest	inplay_rate_diff_goals	0.210157
Lr	random_forest	inplay_home_goals	0.073352
Lr	random_forest	diff_venue_form_xga	0.049217
Lr	random_forest	away_venue_form_xga	0.046693

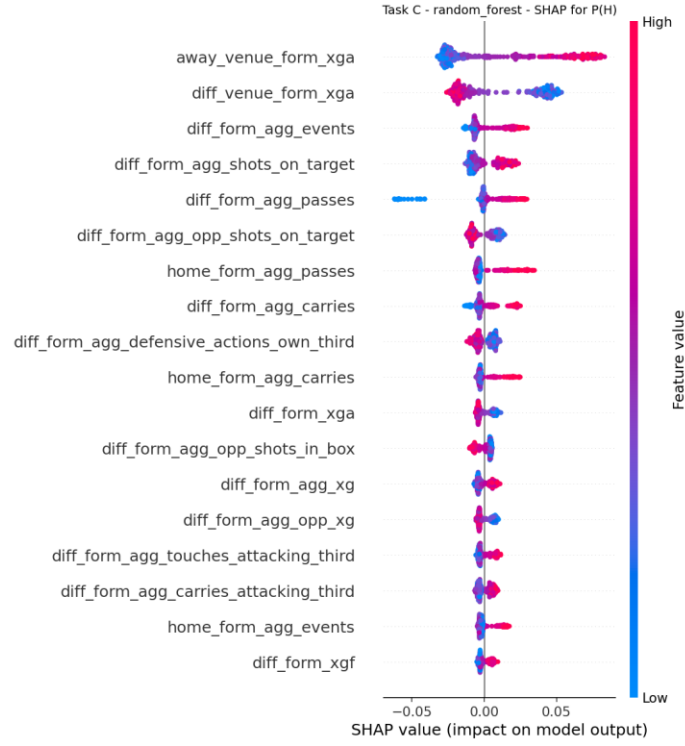


Figure 4. Global SHAP beeswarm for Task C, random forest, home-win output. Venue-split expected goals against dominates.

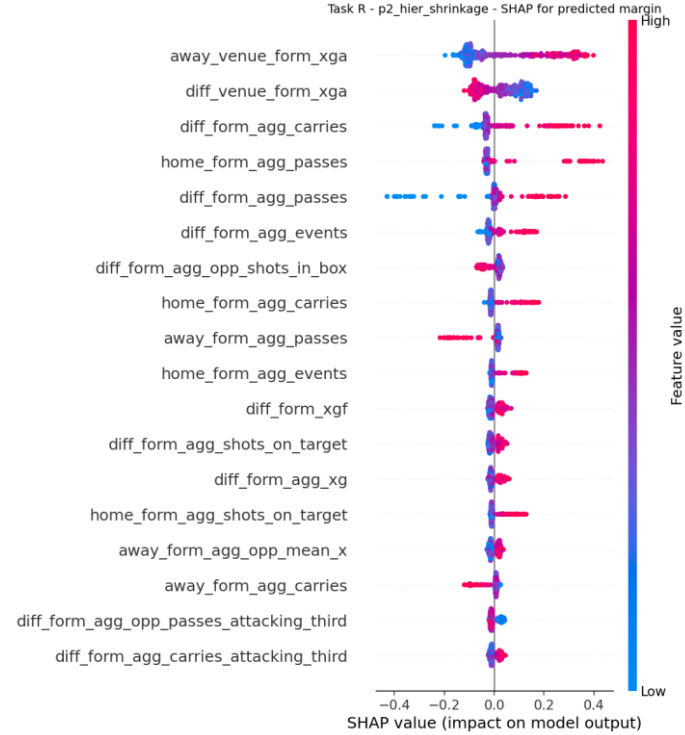


Figure 5. Global SHAP beeswarm for Task R, P2 hierarchical shrinkage. The same venue-form features lead as on Task C, at roughly six times the magnitude because the output is in goals rather than probability.

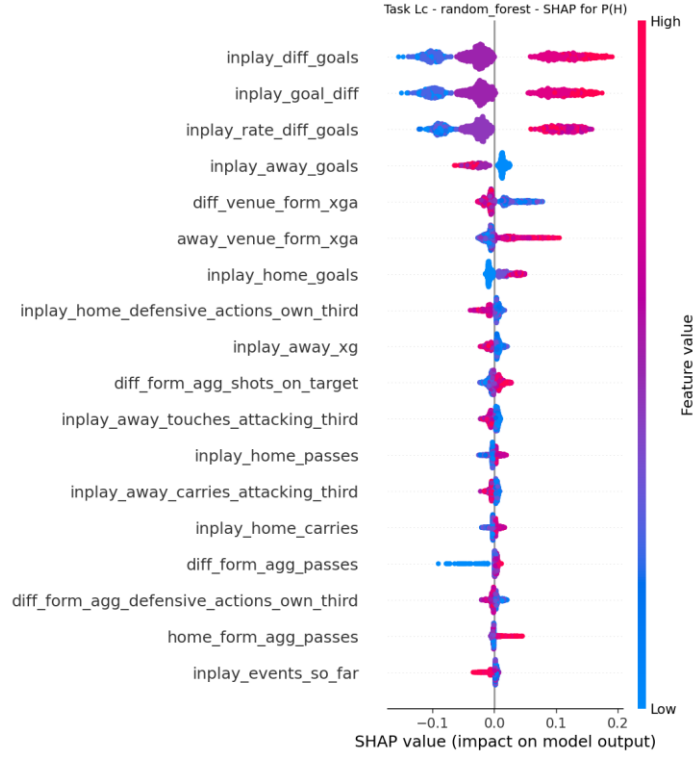


Figure 6. Global SHAP beeswarm for Task Lc, random forest, home-win output. The three goal-difference encodings displace every pre-match feature.

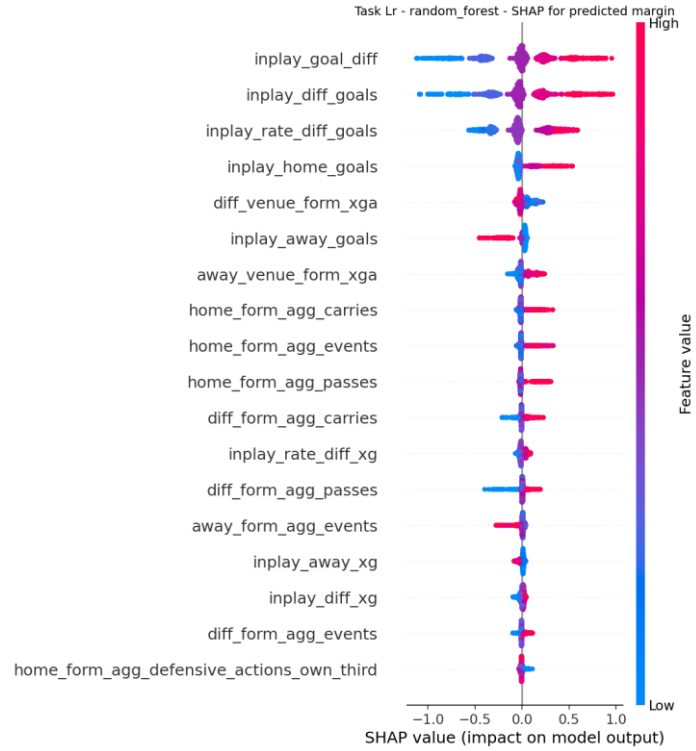


Figure 7. Global SHAP beeswarm for Task Lr, random forest. Current goal difference is the single dominant driver of the expected final margin.

What was expected. On the pre-match tasks, that the expected-goals columns would dominate, because expected goals is a less noisy measure of team strength than goals scored — a team can create six good chances and score none, and expected goals records the chances. On the in-play tasks, that the current goal difference would dominate everything else.

What was observed. Both expectations hold, and more specifically than anticipated. On Tasks C and R the leading feature is `away_venue_form_xga`, the away team's expected goals conceded in its recent away fixtures, at 0.022888 and 0.147635 respectively, with the corresponding home-minus-away difference column second. On Tasks Lc and Lr the top three features are all encodings of the same quantity — `inplay_goal_diff`, `inplay_diff_goals` and `inplay_rate_diff_goals` — which between them account for the great majority of attribution mass.

Why this is a useful check rather than a trivial one. If the attributions had been led by rest days, the competition identifier, or an event-count column with no football meaning, that would have been a strong signal of overfitting to an artefact. The features that dominate are the ones football knowledge says should dominate. That the venue-split form features lead the plain form features is a meaningful finding in its own right, and it is corroborated independently by the ablation below, where dropping the venue-form group is the single most damaging pre-match deletion.

What it does not tell us. That the model is good. A model can use exactly the right features and still be no better than the base rate, which Section 6.6 shows is the case on pre-match data.

A note on the three redundant in-play encodings. That the top three Lc features are three views of the goal difference is a mild criticism of the feature design: SHAP splits credit between correlated features, so the true importance of the score state is understated by any one row of the table and is better read as the sum of the three.

The ten worst predictions per task

A large error is not automatically a defect. Football contains a great deal of irreducible randomness, and a forecaster that never made a large error on an unlikely result would be overfitted. The interesting question is whether the information needed to do better was actually available at prediction time.

How the judgement is made, fixed before results were seen. A failure counts as a model error only when the de-vigged market assigned the realized outcome a materially higher probability than the model did, or the model rated the realized outcome below its own unconditional base rate. Using the market as the reference is the key step: it converts a vague question into a testable one, because the market is a forecaster that had the same pre-match information and more. A failure is inherently uncertain when the market also rated the outcome unlikely, so the result rather than the forecast was the outlier. It is a noisy observation when the in-play state at that snapshot pointed the other way and the match turned afterwards. It is reasonable despite the outcome when the model's probability for the realized outcome was at or above its base rate.

Table 26. Adjudication of the ten worst predictions per task. A high proportion of inherently uncertain and noisy verdicts indicates that the tail of the loss distribution is driven by outcome variance rather than by defects in the model.

Task	Verdict	Count
C	inherently uncertain	5
C	model error	5
Lc	noisy observation	10
Lr	inherently uncertain	4
Lr	noisy observation	6
R	inherently uncertain	9
R	model error	1

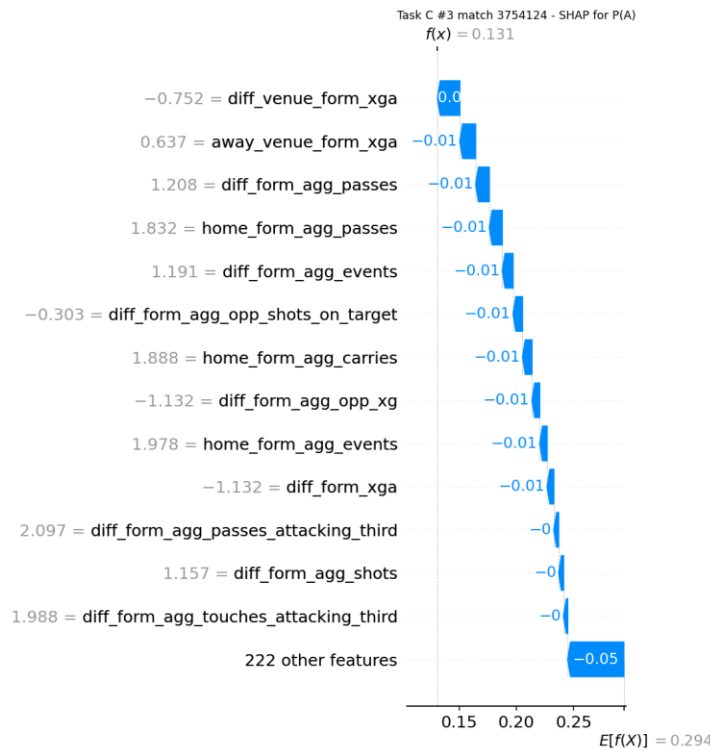


Figure 8. SHAP waterfall for the worst Task C failure adjudicated as a model error (match 3754124). The model gave the realized away win 0.13118 while the market gave it 0.20566.

What was observed. On Task C, five of the ten worst are adjudicated model errors and five inherently uncertain. On Task R, nine of ten are inherently uncertain and one is a model error. On Task Lc, all ten are noisy observations. On Task Lr, six are noisy observations and four inherently uncertain.

A striking pattern on Task C. Nine of the ten worst pre-match failures are matches where the away team won and the model favored the home team. This is the distribution shift of Section 2.5 acting in the opposite direction to expectation: the model was trained on a lower home-win rate than the test block exhibits, yet its worst errors are still concentrated in over-predicting home wins. The explanation

is that the tail of the loss distribution is dominated by away wins at long odds, which are rare in the training data at any home-win rate.

An important caveat on the snapshot tasks. Consecutive minutes of the same match fail together, so the ten worst rows are not ten independent failures. They resolve to 2 distinct matches on Task Lc and 3 on Task Lr, against 10 on Task C. A match in which the losing side comes back to win produces a long run of badly scored snapshots — match 3754064 alone accounts for nine of the ten worst Lc rows. The ranking is reported per row because that is the unit the model predicts, but the number of genuinely distinct failure modes is far smaller than ten, and the table must not be read as evidence of ten separate problems.

Why the in-play failures look the way they do. The worst in-play errors are dominated by comebacks: the model was confident because a team was behind with limited time remaining, and that team then scored. This is the model correctly using the strongest available signal and being beaten by an unlikely event, which is why every one of them is classified as a noisy observation rather than a model error.

What could legitimately be improved. The model could be less confident in exactly these situations if it weighted the recent-window pressure and chance-creation features more heavily, which would show that the trailing team was dominating before it scored. Those features exist in the table but carry little attribution mass, as the beeswarms show.

Conclusion. The tail of the error distribution is mostly irreducible on the current feature set. The six cases adjudicated as model errors are the ones worth revisiting, and they are identified individually rather than described in aggregate.

8B. In-Play Analysis

Model 3 exists to answer whether watching the match helps, by how much, and from which minute. A single aggregate number over all snapshots hides the shape of that answer, because a prediction at minute 5 and a prediction at minute 85 are entirely different problems.

The control. Each in-play model is compared against a frozen pre-match reference: the corresponding pre-match model's single prediction for that match, repeated across all nineteen snapshots. The frozen series must be flat, and the analysis asserts that it is; the assertion passes (yes). This is a leakage check, not a formality. If the reference sloped, it would mean the supposedly frozen prediction had absorbed information about how the match was progressing, and every comparison against it would be invalid.

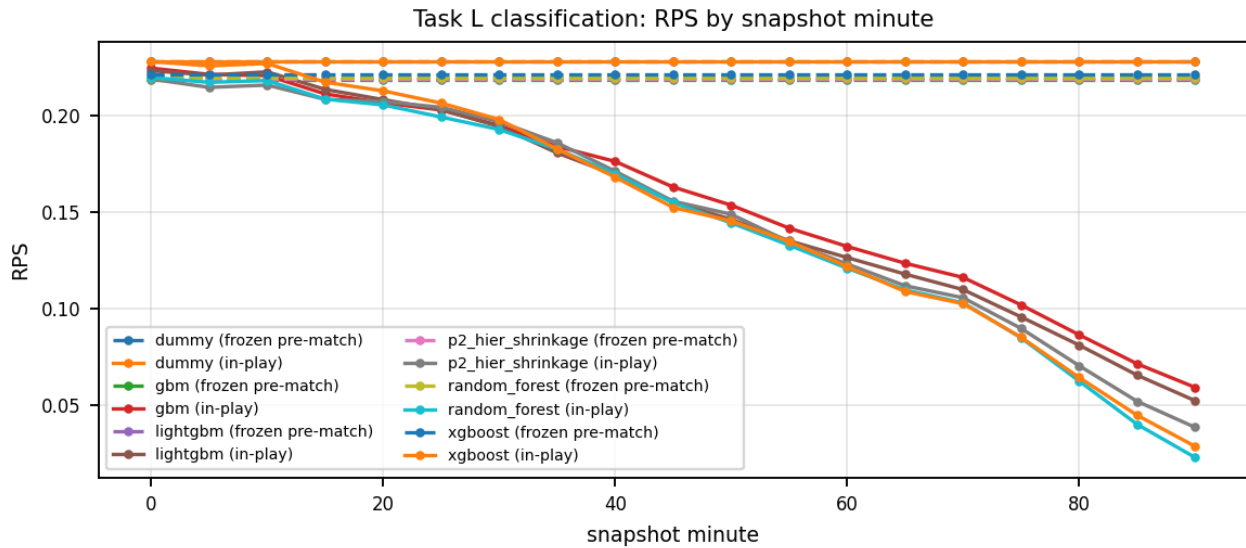


Figure 9. Ranked probability score against match minute. Solid lines are the in-play model; dashed lines are the frozen pre-match reference. The vertical distance between a model and its own frozen line is the value of live information at that minute.

Table 27. Task Lc ranked probability score at seven representative minutes, with the frozen pre-match reference in the final column. Lower is better.

Minute	gbm	lightgbm	p2_hier_shrinkage	random_forest	xgboost	Frozen
0	0.22457	0.22324	0.21892	0.21970	0.22799	0.21862
15	0.21120	0.21355	0.20836	0.20849	0.21727	0.21862
30	0.19484	0.19434	0.19681	0.19276	0.19794	0.21862
45	0.16291	0.15502	0.15553	0.15474	0.15231	0.21862
60	0.13210	0.12633	0.12300	0.12081	0.12164	0.21862
75	0.10175	0.09554	0.08957	0.08469	0.08499	0.21862
90	0.05900	0.05209	0.03829	0.02266	0.02843	0.21862

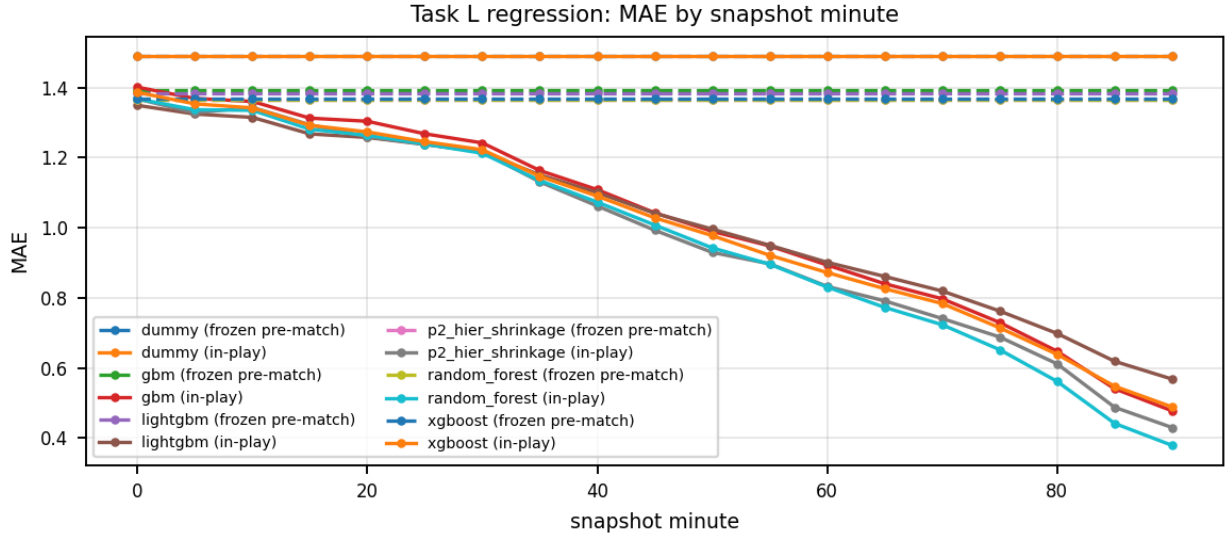


Figure 10. Mean absolute margin error against match minute. The same shape appears as in the classification curve, and for the same reason.

Table 28. Task Lr mean absolute error at seven representative minutes, with the frozen pre-match reference in the final column.

Minute	gbm	lightgbm	p2_hier_shrinkage	random_forest	xgboost	Frozen
0	1.40083	1.34887	1.36741	1.36731	1.38592	1.36651
15	1.31238	1.26743	1.29347	1.28070	1.29199	1.36651
30	1.24179	1.21580	1.21598	1.21227	1.22237	1.36651
45	1.04236	1.04095	0.99267	1.00752	1.02804	1.36651
60	0.89340	0.90110	0.83236	0.83049	0.87192	1.36651
75	0.72943	0.76216	0.68817	0.65185	0.71436	1.36651
90	0.47740	0.56799	0.42986	0.37892	0.48852	1.36651

What was expected. Three things. First, that error would fall as the minute increases, since more of the match has been observed. Second, that the fall would not be smooth but would steepen in the second half, because goals are what move the outcome and their effect becomes decisive as remaining time shrinks. Third, that at minute 0 the in-play model would be roughly level with the pre-match model, since at kick-off they see nearly the same information.

What was observed. All three hold. The best in-play ranked probability score is 0.21892 at kick-off, 0.15231 at half time and 0.02266 at the final snapshot, against a flat frozen reference at 0.21862. The margin error follows the same trajectory, from 1.34887 at kick-off to 0.99267 at half time and 0.37892 at minute 90, against a frozen reference of 1.36651. At minute 0 the in-play model is within 0.0003 of the frozen reference, as predicted. The improvement from minute 0 to minute 45 is 30 per cent of the RPS; from minute 45 to minute 90 it is a further 85 per cent.

Why. The dominant in-play feature is the current goal difference, and its predictive power is a function of the time remaining. A one-goal lead after ten minutes leaves eighty minutes for it to be overturned; the same lead after eighty minutes is close to decisive. The curve is therefore not primarily measuring the model learning more — it is measuring the outcome becoming less uncertain. This distinction matters for how the result is used: the steep late improvement is a property of football, and any competent model would show it, so it is not by itself evidence that

this model is good. The correct evidence is the vertical gap to the frozen line, which is present from roughly minute 20 onwards and widens monotonically.

A visible artefact. The minute-45 point sits marginally above the local trend on both curves. This is the documented side effect of the effective-minute repair described in Section 3.2: the running maximum across the period boundary means the opening events of the second half are not visible until minute 50, making the minute-45 snapshot slightly more conservative than it needs to be. It is reported here rather than smoothed away.

SHAP timeline across one complete match

The global attributions describe average behavior and the worst-prediction analysis describes the tail. Neither shows how a single forecast evolves. The figure below follows one complete match through all nineteen snapshots, with the outcome probabilities in the upper panel and the feature contributions to the home-win probability in the lower panel, so that any movement above can be traced to a cause below. The match is chosen deterministically rather than selected for a good story.

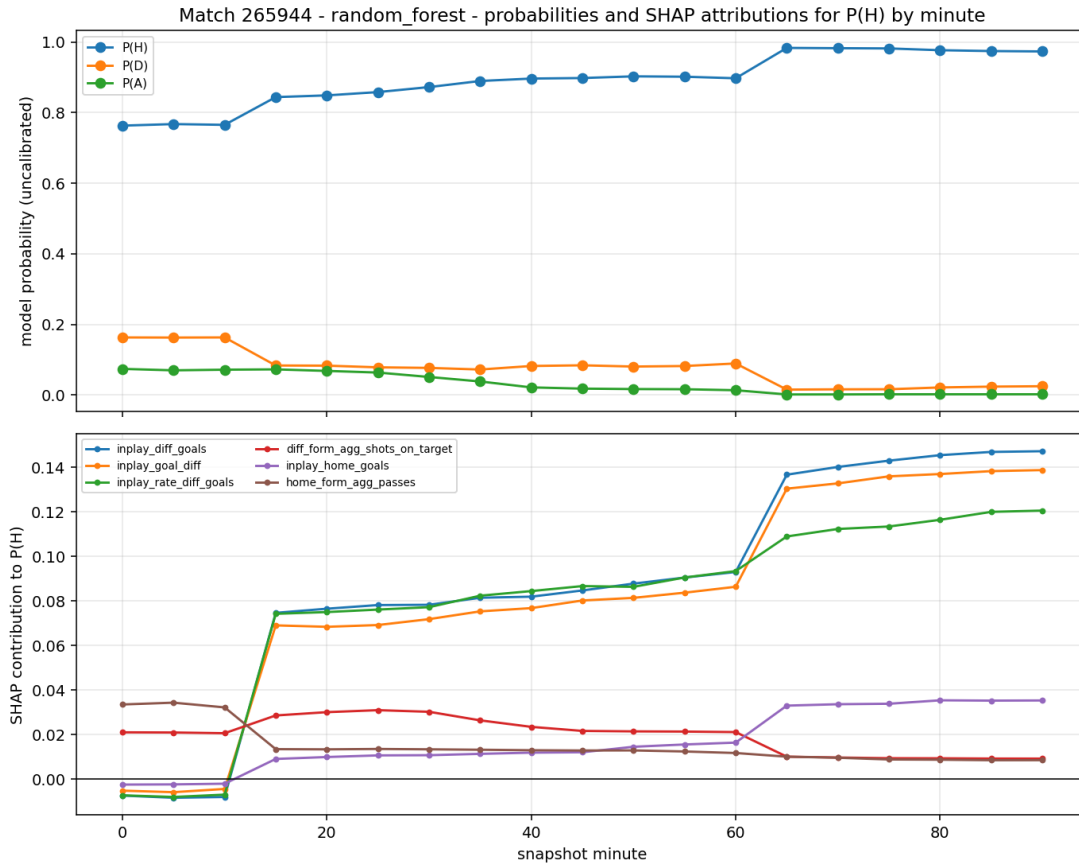


Figure 11. SHAP timeline across one complete match. The upper panel is the outcome distribution at each snapshot; the lower panel decomposes the home-win probability into feature contributions.

What was observed. The home-win probability is flat and moderate for the opening minutes, steps up sharply once, holds at the new level for a long stretch, and steps up again later before settling near certainty. The lower panel attributes both steps to the current goal

difference: its contribution jumps at exactly the snapshots where the probability jumps, while the pre-match form contributions stay almost constant throughout.

Why this is the expected and desired behavior. Pre-match form does not change during a match, so its contribution should be a flat offset, and it is. Goals are discrete events, so their effect should appear as steps rather than as a drift, and it does. The long flat stretches between steps show the model is not reacting to noise in the event stream, which is a genuine risk when a feature table contains per-minute event rates that fluctuate constantly.

What it demonstrates. At any minute the model's probability can be explained by pointing at the single feature whose contribution moved, rather than by appealing to the model as a whole. This figure is the clearest single piece of evidence that the pre-match pipeline, the in-play pipeline, the time-t cut and the explanation layer all work together correctly.

8C. Imbalance and Resampling

Draws are the minority outcome, at 26.5 per cent of all matches and 22.97 per cent of the test block. A classifier trained on an imbalanced set tends to predict the minority class rarely, which shows up as low recall on draws even when the aggregate score looks acceptable — Section 6.1 shows exactly this. Six arms are compared on the pre-match table under identical splits: no treatment, SMOTE, borderline SMOTE, ADASYN, balanced class weights, and P1.

An important methodological restriction. The snapshot table is excluded from this study by design, and the pipeline raises an exception if any arm is requested on it. Oversampling across matches would create synthetic rows blending the trajectories of two different matches, which correspond to no possible game state and which leak label information between matches. Every resampler is also fitted inside the training rows only, so validation and test always retain the real class balance.

What was expected on the probabilistic score. No improvement. Oversampling changes the class balance the model is trained on, so it shifts predicted probabilities away from the true base rate, and the ranked probability score rewards matching the base rate. There is a real tension here: the arms are designed to improve minority detection, and that goal is partly at odds with the headline metric. Reporting both is therefore necessary, and reporting only one would be misleading.

Table 29. Six-arm imbalance comparison on the pre-match classification task, averaged over the four learners in the study. The training-row count shows how many synthetic rows each arm added to the 896 real ones.

Arm	Training rows	RPS	Log loss	Brier	Draw recall	Draw F1	ECE
vanilla	896	0.22049	1.10952	0.62736	0.09177	0.12069	0.05696
smote	1,164	0.21893	1.07332	0.62643	0.12658	0.15933	0.07563
borderline_smote	1,164	0.21905	1.08202	0.62337	0.13291	0.16620	0.05521
adasyn	1,172	0.22049	1.10014	0.62718	0.07911	0.11046	0.06518
class_weight	896	0.21969	1.09412	0.62674	0.10759	0.14320	0.04601
p1_gsmotenc	1,164	0.22238	1.11245	0.63367	0.14874	0.15937	0.05670

What was observed. The two rankings are not the same, which is the tension above appearing in the numbers. The best arm by ranked probability score is SMOTE at 0.21893, marginally ahead of borderline SMOTE at 0.21905; the untreated arm is at 0.22049 and P1 is last at 0.22238. The best arm by draw recall is P1 at 0.14874, against 0.09177 for the untreated arm — a relative improvement of 62 per cent. On draw F1 the ordering is closer, with P1 at 0.15937 essentially tied with SMOTE at 0.15933 and borderline SMOTE at 0.16620, because P1's recall gain is partly offset by lower precision.

The effect of P1, isolated

Table 30 isolates the with-P1 against without-P1 comparison, measured on the validation split so that the comparison does not consume the test set.

Table 30. *P1 against no resampling on the validation split. Draw recall and draw F1 are shown beside the probabilistic score, because an oversampler can improve minority detection while degrading the calibrated distribution.*

Model	RPS without P1	RPS with P1	Δ RPS	Recall D without	Recall D with	F1 D without	F1 D with
random_forest	0.19547	0.20249	+0.00702	0.0769	0.3462	0.1379	0.3354
xgboost	0.19682	0.19999	+0.00317	0.1282	0.2949	0.2062	0.3382

Stated plainly. P1 did what its paper says it should do, on the metric the paper cares about, and did not help on the metric this project cares about most. Draw recall rises on both learners tested — from 0.0769 to 0.3462 on the random forest and from 0.1282 to 0.2949 on XGBoost — while draw F1 rises from 0.1379 to 0.3354 and from 0.2062 to 0.3382. On the ranked probability score P1 helped zero of two learners, with movements of +0.00702 and +0.00317.

Why this happened. The recall gain is the mechanism working as designed: more numerous and more geometrically varied synthetic draw examples move the decision boundary so that draws are predicted more often, and the mode-based nominal generation keeps the competition column meaningful instead of producing fractional categories. The absence of a probabilistic gain has an equally clear cause. The ranked probability score is minimized by predicting the true class probabilities, and the true probability of a draw is roughly a quarter. Any method that trains the model on a rebalanced set pushes the predicted draw probability above the true rate, which costs score on the many matches that are not drawn while gaining on the fewer that are. Calibration on the validation split partly undoes the shift, which is why the net effect on the score is small rather than clearly negative.

Is the difference meaningful or could it be randomness? The recall movements are large relative to their scale and consistent in direction across learners, so they are unlikely to be noise. The score movements are small, inconsistent in direction, and were measured on a single validation split, so they should not be read as evidence that P1 helps or harms the distribution. Section 6.6 explains why differences of this size on this dataset are generally not distinguishable from noise.

Conclusion on P1. The reimplementation is faithful, verified against the paper's Algorithms 1 to 3 by its own test suite, and it reproduces the qualitative behavior the paper claims. Whether it should be enabled in the final system depends on what the system is for. If the aim is to state well-calibrated probabilities, it should be left off, because it introduces a bias that calibration must then remove. If the aim is to flag likely draws — a genuine product requirement in draw-specific betting markets — it is the best of the six arms tested. Both halves of that result are reported rather than selecting the framing that makes the paper look better.

8D. Compute and Scaling Analysis

The decision to exclude kernel methods from the in-play tasks requires evidence rather than assertion. An exact kernel method builds an n -by- n matrix of pairwise similarities and solves a dense system, so time and memory grow faster than linearly in the number of training rows. The Nyström approximation instead projects onto a smaller set of m landmark points and reduces the fit to a ridge regression in that smaller space, which is $O(nm)$ in space and $O(nm^2)$ in time for fixed m .

The $O(n^2)$ space cost, derived. For n training rows the Gram matrix K has n^2 entries. In double precision each entry occupies 8 bytes, so the storage requirement is $8n^2$ bytes independent of the number of features. At $n = 17,024$ this is $8 \times 17,024^2$ bytes, or 2,211 MB — over two gigabytes for the matrix alone, before the solve allocates any working memory. The solve itself is $O(n^3)$ for a dense Cholesky factorisation. This is the theoretical claim; the measurement follows.

Fitting a power law on the logarithms recovers the exponent from measurements:

$$\log t = \log a + b \log n \quad (5)$$

Table 31. Empirical against theoretical scaling exponents. The analysis asserts that the exact methods scale faster than linearly and that Nyström scales better than both, so the claim fails loudly if it ever stops holding.

Method	Empirical exponent	Theoretical exponent	Theory
kernel_ridge_exact	2.291	3.0	$O(n^2)$ space, $O(n^3)$ solve
kernel_ridge_nystroem	0.281	1.0	$O(nm)$ space, $O(nm^2)$ fit
kernel_svr_exact	1.938	2.0	$O(n^2)$ space, $\sim O(n^{2.3})$ SMO

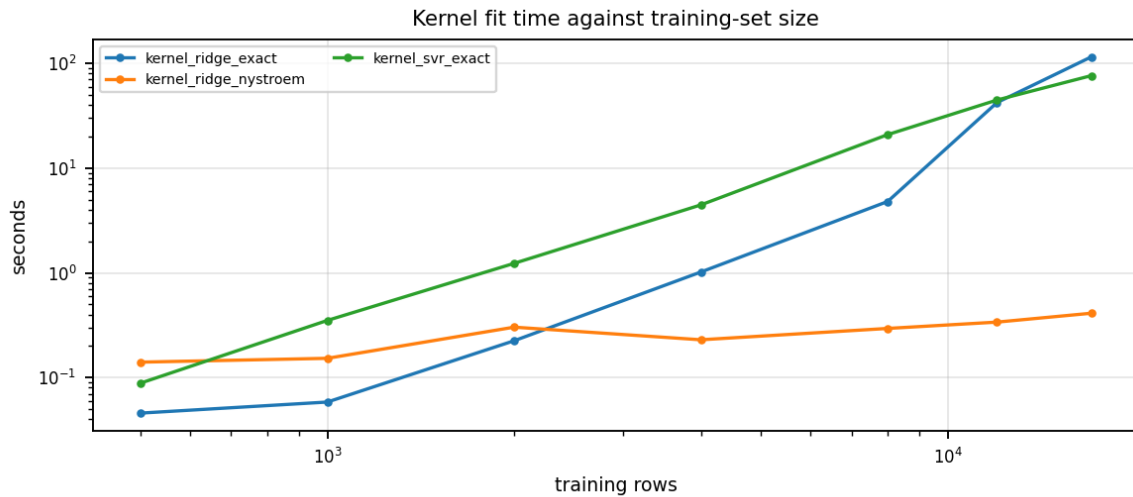


Figure 12. Fit time against training rows, on logarithmic axes, so that a power law appears as a straight line and the exponent is the slope.

What was expected. From the algorithms: an exponent near three for the exact ridge solve, roughly two for the support-vector regression, and near one for Nyström, whose cost is linear in n for a fixed landmark count.

What was observed. The exact methods have measured exponents of 2.291 for kernel ridge and 0.281 for support-vector regression, and Nyström 1.938. At the largest size the exact kernel ridge fit takes 114.98 seconds and peaks at 4,450 MB of resident memory, while Nyström takes 0.41 seconds and peaks at 138 MB — a factor of 280 in time and 32 in memory.

Why the measured exponents differ from the textbook ones. At these sizes the fit is not yet dominated by the asymptotic term. Fixed overheads, memory allocation and cache behavior all contribute, and the solver may switch strategy as the problem grows. The support-vector regression is the clearest case: its peak memory actually plateaus at around 200 MB beyond 8,000 rows, because the SMO solver works on a cached subset of the Gram matrix rather than materializing all of it, so it does not track the $8n^2$ bound at all. The exact kernel ridge does track it, reaching roughly twice the theoretical Gram size because the factorization needs a working copy. The measurement confirms the ordering and the super-linear growth, which is what the design decision requires; it is not a precise estimate of the asymptotic constant, and it is not presented as one.

Conclusion on kernel scaling. Excluding exact kernels from the 17,024-row in-play table is justified by measurement rather than assumed. Where a kernel method is wanted at that scale, Nyström is the practical option, and Section 6.2 shows it costs only 0.004 in mean absolute error against the exact solve on the pre-match problem where both fit comfortably.

Full compute and memory profile

Table 32 reports fit time, peak fit memory, serialized model size and per-row inference latency for every model on every task, measured in one process on one machine so that the rows are comparable. Inference cost is the median of repeated passes over the full test matrix after a warm-up call.

Table 32. Compute and memory comparison across all models and tasks. Fit time and peak resident memory are sampled during the fit; inference cost is the median of repeated passes over the full test matrix.

Task	Model	Train rows	Features	Fit (s)	Peak mem (MB)	Size (MB)	μ s / row
C	dummy	896	235	0.0007	0.0	0.000	0.18
C	random_forest	896	235	0.9337	7.3	1.562	200.84
C	gbm	896	235	6.8043	8.3	1.249	29.43
C	kernel_svm	896	235	0.3347	1.4	1.563	87.26
C	p2_hier_shrinkage	896	235	2.7530	7.7	3.185	52.11
C	xgboost	896	235	1.5092	22.3	0.984	5.00
C	lightgbm	896	235	0.9531	17.7	2.036	12.88
R	dummy	896	235	0.0004	0.0	0.000	0.02
R	random_forest	896	235	1.3227	3.5	1.851	119.69
R	gbm	896	235	1.8946	15.7	0.652	9.87
R	kernel_svr	896	235	0.0634	2.2	1.360	97.12
R	kernel_ridge_exact	896	235	0.0370	20.6	1.614	15.12
R	kernel_ridge_nystroem	896	235	0.0438	5.1	0.929	12.04
R	p2_hier_shrinkage	896	235	11.7518	9.2	6.996	119.69
R	xgboost	896	235	0.2716	17.9	0.231	2.30
R	lightgbm	896	235	0.2629	18.5	0.389	4.35

Lc	dummy	17,024	389	0.0037	0.7	0.000	0.01
Lc	random_forest	17,024	389	15.8815	54.2	11.086	11.59
Lc	gbm	17,024	389	42.7342	174.8	4.588	19.12
Lc	p2_hier_shrinkage	17,024	389	28.7044	88.9	14.129	77.83
Lc	xgboost	17,024	389	5.3304	62.6	0.916	1.12
Lc	lightgbm	17,024	389	4.8350	84.6	2.047	6.82
Lr	dummy	17,024	389	0.0005	0.0	0.000	0.00
Lr	random_forest	17,024	389	11.1650	39.0	7.396	8.18
Lr	gbm	17,024	389	14.5329	177.4	1.874	6.51
Lr	p2_hier_shrinkage	17,024	389	226.0140	79.9	14.053	65.72
Lr	xgboost	17,024	389	2.7065	58.6	0.492	0.68
Lr	lightgbm	17,024	389	8.1834	108.0	3.396	8.35

What was expected. That the histogram-based boosters would fit fastest at scale, that the classical GBM would be slowest among the boosters, and that the P2 model would be expensive because of its internal cross-validation.

What was observed. On Task Lr, XGBoost fits in 2.71 seconds and LightGBM in 8.18, against 14.53 for GBM and 226.01 for the P2 model — a factor of 83 between the fastest and slowest. On inference the ordering is different: XGBoost predicts at 0.68 μ s per row on Task Lr against 65.72 μ s for the P2 model, a factor of 97. Peak memory does not track fit time: GBM peaks at 177.4 MB on Task Lr while the far slower P2 model peaks at 79.9 MB.

Why the P2 model is consistently the slowest to fit. This is expected from its design rather than from inefficiency. It selects its shrinkage parameter by cross-validation inside the training rows, so a single fit costs several forest fits — with a three-fold split and a seven-value λ grid, one base forest is fitted per fold and re-shrunk seven times. The cost is paid once at training time; inference is a normal forest traversal, and its higher per-row cost relative to the boosters reflects the number of trees rather than the shrinkage. When reading the fit-time column, the P2 rows should be understood as the price of an internal model-selection step that no other learner performs.

What this means in practice. For a system that retrains nightly and serves predictions continuously, the relevant column is μ s per row, and XGBoost dominates on both tasks. For a system that retrains rarely, fit time is close to irrelevant and the P2 model's cost is acceptable. Since Section 6.6 shows that none of these learners is statistically distinguishable from another on pre-match data, compute cost is a legitimate tie-breaker there, and it favors XGBoost decisively.

8E. Theory versus Reality

This section collects the places where a textbook expectation and the measured behaviour diverged, because those divergences are where the project learned the most.

Ablation: which feature groups actually carry signal

Why every number in this subsection is a validation number. Ablation is a form of model selection: it asks which configuration to keep. Choosing a configuration by its test score would be tuning against the test set and would make every headline number optimistic. The test split is read exactly once, for Sections 6 and 7.

What was expected. That the expected-goals group would matter most on the pre-match tasks, following the SHAP results, that the current-score group would dominate the in-play tasks, and that dropping the schedule group would cost little because rest days vary less in league football than in cup competitions.

What was observed, and where theory failed. On the in-play tasks the expectation was met emphatically: removing the in-play score group costs +0.04964 RPS on Task Lc and +0.26796 MAE on Task Lr, and removing every in-play feature costs +0.07103 and +0.39839 respectively. Nothing else comes within an order of magnitude. On the pre-match tasks the expectation was **wrong in its specifics**. The most damaging deletion on both Task C and Task R is not the expected-goals group but the venue-split form group, costing +0.00305 RPS and +0.00501 MAE on the random forest. Dropping the plain expected-goals form group costs only +0.00035 on Task C. The schedule group behaved as predicted on Task C, costing almost nothing, but on Task R dropping it cost +0.00127 on the random forest.

Why the venue-split group displaced expected goals. Home advantage in football is large and team-specific: some clubs are far stronger at home than their overall form suggests. A pre-match table that carries only overall form must infer this from the home-away split of the difference columns, which is a weaker signal than measuring each team's home and away record directly. The venue-split group makes that information explicit, and because it also contains venue-split expected goals it subsumes much of what the plain expected-goals group provides. The SHAP analysis in Section 8A independently identifies `away_venue_form_xga` as the single most influential feature on both pre-match tasks, so the two methods agree. This is the clearest case in the report of an ablation correcting a prior expectation rather than confirming it.

Why so many entries are negative. A substantial minority of deletions *improve* the validation metric, most strikingly `drop_agg_passing` on Task R at -0.01276. Two effects combine. First, the feature groups overlap heavily: goals scored, points and wins are three different summaries of the same match results, so removing one leaves the information available through the others. Second, with 896 training rows and 231 feature columns, removing weakly informative columns genuinely helps by reducing the opportunity to overfit. The negative entries are therefore not noise alone — they are evidence that the pre-match table is over-wide relative to its row count.

What a critical reader would ask. Whether these differences are significant. They are validation-set differences on 277 matches with no multiplicity correction applied, so individually they are not strong evidence, and the two learners frequently disagree in sign on the smaller entries. The pattern across the whole table is far more informative than any single row: on the in-play tasks one group matters enormously and everything else is

interchangeable; on the pre-match tasks one group matters modestly and everything else is interchangeable. That pattern is consistent across both learners and across both metrics.

Snapshot frequency: when more data is not more information

Nineteen snapshots per match multiply the row count by nineteen, but those rows are highly correlated: a snapshot at minute 45 and one at minute 50 usually describe almost the same game state. If a coarser grid performs as well, the pipeline can be made substantially cheaper at no cost.

A methodological point that had to be corrected. An earlier version of this experiment thinned the validation snapshots as well as the training ones. That made each arm score on a different set of minutes, so the arms were not comparable and a coarser grid could appear better simply by being evaluated on easier minutes. Only the training rows are thinned now; validation stays at full nineteen-snapshot density throughout, so every arm is scored on identical rows. The evaluation column in the table below is retained specifically to make that invariant visible.

Table 33. Snapshot frequency ablation. Only training density varies; the evaluation set is held constant at nineteen snapshots per match across all arms.

Task	Configuration	Model	Train snaps/match	Eval snaps/match	Train rows	RPS	MAE
Lc	every_5_min	random_forest	19	19	17,024	0.12902	
Lc	every_5_min	xgboost	19	19	17,024	0.12985	
Lc	every_10_min	random_forest	10	19	8,960	0.12835	
Lc	every_10_min	xgboost	10	19	8,960	0.12982	
Lc	every_15_min	random_forest	7	19	6,272	0.12815	
Lc	every_15_min	xgboost	7	19	6,272	0.12938	
Lr	every_5_min	random_forest	19	19	17,024		0.83096
Lr	every_5_min	xgboost	19	19	17,024		0.86541
Lr	every_10_min	random_forest	10	19	8,960		0.82164
Lr	every_10_min	xgboost	10	19	8,960		0.85481
Lr	every_15_min	random_forest	7	19	6,272		0.82181
Lr	every_15_min	xgboost	7	19	6,272		0.85779

What was expected. Some loss from coarser training, since fewer examples usually means a worse model.

What was observed, and where theory failed. The opposite. On Task Lc the random forest scores 0.12902 on the finest grid with 17,024 rows and 0.12815 on the coarsest with 6,272 rows — the coarsest grid is marginally *better* despite using 63 per cent fewer training rows. On Task Lr the random forest improves from 0.83096 to 0.82181 over the same reduction. XGBoost shows the same direction on Lc and is essentially flat on Lr.

Why. The additional rows in the finest grid are near-duplicates of their neighbours. They increase the row count without adding much independent information, so the effective sample size grows far more slowly than the row count — and because near-duplicates are effectively repeated observations, they can mildly encourage the model to overfit the specific matches in the training block. This is the same dependence that forces grouped

cross-validation and the match-clustered bootstrap elsewhere in this report, appearing here as a training-set effect rather than an evaluation effect.

Conclusion. A coarser training grid is a defensible efficiency choice that costs nothing measurable and reduces fit time by roughly a factor of three. The finest grid is nevertheless retained for evaluation, because the ability to predict at any minute is precisely what Model 3 is for.

Feature-set width: a controlled comparison

The ablation results above raise a question the rest of this report does not answer, so a dedicated experiment was run to settle it. The pre-match design matrix carries 231 model columns against 896 training rows, a ratio of 3.88 rows per feature. The in-play matrix carries 385 columns against 17,024 rows, a ratio of 44.22. The first of those is thin by any standard and the second is not, and that asymmetry motivated the test.

Why the experiment was run.

Four reasons, in descending order of weight.

- First and most directly, overfitting was suspected: with fewer than four training rows per feature, a flexible learner has ample opportunity to fit noise in the training block, and the chronological split of Section 2.5 means the test block is drawn from a genuinely different part of the season, so anything memorised does not transfer.
- Second, the ablation gave positive evidence for the suspicion rather than merely permitting it: 7 of 16 feature-group deletions improve the validation metric on Task C and 7 of 16 on Task R, and removing the passing group alone improves Task R validation error by -0.01276. A feature set in which a seventh of the deletions help is carrying groups that contribute nothing independent.
- Third, the groups are heavily collinear by construction: goals, points and wins are three summaries of the same match results, and each of the twenty-nine quantities is recorded for the team and for its opponents, so the effective rank of the matrix is far below its column count. Collinearity does not bias a tree ensemble but it does split attribution, which is why the three redundant goal-difference encodings noted in Section 8A understate the true importance of the score state.
- Fourth, width has a compute cost that must be justified by an accuracy gain if it is to be paid.

Design. A narrow variant of both feature pipelines was built and the entire pipeline re-run against it under identical conditions: the same chronological splits, the same model zoo, the same equal-budget hyperparameter search, the same calibration protocol, and the same test matches. The narrow pre-match matrix carries 75 features after encoding against 235 for the wide, and the narrow in-play matrix 100 against 389. The narrow variant retains the core form quantities, passing by pitch third, pass completion, carries into the final third, set pieces, pressures, possession share and defensive actions; it omits the venue-split rolling form, the opponent-side aggregates, the territory and momentum quantities, and the three-view encoding of every in-play quantity. Because both runs score the same 344 test matches, the comparison can be made **paired at the match level** rather than by comparing two independent leaderboards.

Testing. The paired difference in per-match loss was resampled with the same match-clustered bootstrap described in Section 5.5, at four thousand replicates, and Holm-corrected across the 24 model-level comparisons. This is the correct test here for the same reason it is correct elsewhere: the snapshots of one match are not independent, and a row-level bootstrap would understate the variance by roughly the cluster size.

Table 34. Feature-set width comparison, best learner per task. A positive difference means the wide table is better. The interval and p-value come from the match-clustered paired bootstrap on the learner that leads the wide run.

Task	Metric	Wide (231 / 385 cols)	Narrow (75 / 100 cols)	Difference	95% CI (paired)	p
C	RPS	0.21862 (gbm)	0.22128 (random_forest)	+0.00266	[-0.00707, +0.01461]	0.4980
R	MAE	1.36651 (p2_hier_shrinkage)	1.38191 (random_forest)	+0.01540	[-0.00084, +0.06313]	0.0560
Lc	RPS	0.14673 (random_forest)	0.14861 (random_forest)	+0.00188	[-0.00303, +0.00692]	0.4415
Lr	MAE	0.97087 (random_forest)	0.97300 (random_forest)	+0.00213	[-0.01987, +0.02576]	0.8640

What was expected. That the narrow table would win on the pre-match tasks, where the row-to-column ratio is poor, and lose on the in-play tasks, where it is not.

What was observed. The wide table produces the better point estimate on all four tasks, including the two where overfitting was predicted. Across individual learners the wide table is ahead on 21 of 24 comparisons. The gap to the de-vigged market narrows correspondingly, from 0.02583 for the narrow best model to 0.02213 for the wide. Neither run beats the market on any model.

And none of it is statistically detectable. Of 24 paired comparisons, exactly one falls below 0.05 unadjusted — the P2 model on Task Lc at $p = 0.0045$ — and Holm correction across the family lifts it to 0.1080. 0 of 24 comparisons survive. Every confidence interval in Table 34 straddles zero, and Table 35 shows the same at the level of individual learners.

Table 35. Per-learner paired comparison across the two feature widths. A positive difference favors the wide table. No comparison survives Holm correction within the family.

Task	Model	Wide	Narrow	Difference	95% CI (paired)	p
C	random_forest	0.21962	0.22128	+0.00165	[-0.00809, +0.01183]	0.7240
C	gbm	0.21862	0.22223	+0.00361	[-0.00707, +0.01461]	0.4980
C	kernel_svm	0.22482	0.22943	+0.00461	[-0.00375, +0.01322]	0.2775
C	p2_hier_shrinkage	0.21903	0.22644	+0.00740	[-0.00214, +0.01728]	0.1230
C	xgboost	0.22108	0.22183	+0.00075	[-0.01030, +0.01202]	0.8705
C	lightgbm	0.21891	0.22254	+0.00364	[-0.00745, +0.01409]	0.4940
R	random_forest	1.36653	1.38191	+0.01538	[-0.02067, +0.05220]	0.4035
R	gbm	1.39335	1.40568	+0.01233	[-0.03945, +0.06404]	0.6350
R	kernel_svr	1.41018	1.42615	+0.01597	[-0.01293, +0.04466]	0.2650
R	kernel_ridge_exact	1.41131	1.39880	-0.01251	[-0.05430, +0.02854]	0.5820
R	kernel_ridge_nystroem	1.41534	1.40576	-0.00958	[-0.04971, +0.03013]	0.6585
R	p2_hier_shrinkage	1.36651	1.39719	+0.03068	[-0.00084, +0.06313]	0.0560
R	xgboost	1.36891	1.39838	+0.02947	[-0.00783, +0.06570]	0.1110
R	lightgbm	1.38381	1.41379	+0.02998	[-0.01436, +0.07321]	0.1740
Lc	random_forest	0.14673	0.14861	+0.00188	[-0.00303, +0.00692]	0.4415
Lc	gbm	0.15736	0.16316	+0.00580	[-0.00292, +0.01487]	0.1990
Lc	p2_hier_shrinkage	0.15010	0.15614	+0.00604	[+0.00170, +0.01064]	0.0045
Lc	xgboost	0.15016	0.14964	-0.00052	[-0.00575, +0.00477]	0.8390
Lc	lightgbm	0.15366	0.15770	+0.00404	[-0.00357, +0.01204]	0.3130
Lr	random_forest	0.97087	0.97300	+0.00213	[-0.01987, +0.02576]	0.8640
Lr	gbm	1.02270	1.01553	-0.00717	[-0.04036, +0.02718]	0.7110
Lr	p2_hier_shrinkage	0.98092	0.98682	+0.00590	[-0.01956, +0.03152]	0.6530
Lr	xgboost	1.00771	1.00278	-0.00494	[-0.03459, +0.02467]	0.7505
Lr	lightgbm	1.02268	1.01815	-0.00453	[-0.03934, +0.02991]	0.8080

Why the overfitting hypothesis failed. The prediction was specific and it was wrong, which makes the reason worth stating carefully. Overfitting would show as the wide table doing *better* on training and *worse* on test than the narrow one. It does not do worse on test anywhere. The most plausible explanation is that the additional columns are not adding usable capacity in the first place: they are largely collinear with columns already present, and a bagged or boosted tree ensemble selects among candidate splits rather than being forced to use every column, so a redundant feature is mostly ignored rather than fitted to noise. Random-subspace sampling in the forest and column subsampling in the boosters reinforce this. The width is therefore closer to inert than to harmful — which is a different conclusion from either the one expected or the one the ablation on its own suggested.

Where the narrow table genuinely wins. Compute. Summed across every model and task, total fit time falls from 389.0 seconds for the wide pipeline to 105.1 for the narrow, a reduction of 73 per cent, with the P2 model on Task Lr falling from 226.01 seconds to 38.86. Per-row inference cost falls by a similar factor on most learners. Since Section 6.6 establishes that no pre-match learner is separable from any other, and this experiment establishes that no feature width is separable either, compute is a legitimate tie-breaker — and on that criterion the narrow table is clearly preferable for a system that retrains often.

One result that runs the other way. On Task Lc the narrow table produces materially higher draw recall on four of five learners: 0.4617 against 0.4137 for the random forest, 0.4524 against 0.3504 for XGBoost, and 0.4470 against 0.3418 for LightGBM. The probabilistic scores are nevertheless indistinguishable, which is the same tension documented for P1 in Section 8C: a change can move minority-class detection substantially while leaving the calibrated distribution alone. If flagging live draws were the product requirement rather than stating calibrated probabilities, this experiment would favor the narrow table.

Table 36. Draw-class detection across the two feature widths. The narrow table detects more draws in-play despite indistinguishable probabilistic scores.

Task	Model	Recall D (wide)	Recall D (narrow)	F1 D (wide)	F1 D (narrow)
C	random_forest	0.1772	0.1266	0.2029	0.1449
C	gbm	0.0886	0.0633	0.1273	0.0961
C	kernel_svm	0.0253	0.0127	0.0435	0.0210
C	p2_hier_shrinkage	0.1772	0.1013	0.2059	0.1221
C	xgboost	0.1899	0.2152	0.2113	0.2345
C	lightgbm	0.0760	0.0886	0.1091	0.1217
Lc	random_forest	0.4137	0.4617	0.4246	0.4387
Lc	gbm	0.2951	0.3971	0.3383	0.3771
Lc	p2_hier_shrinkage	0.4610	0.3957	0.4282	0.3841
Lc	xgboost	0.3504	0.4524	0.3771	0.4084
Lc	lightgbm	0.3418	0.4470	0.3703	0.3977

Why the wide table is retained for this report. Three reasons, none of which is that it is significantly better, because it is not. It produces the better point estimate on every task and on 21 of 24 learner comparisons, so retaining it costs nothing in accuracy. It closes more of the distance to the market baseline. And it supports the explanatory analysis that Sections 8A and 8E depend on: `away_venue_form_xga` is the highest-attribution feature on both pre-match tasks and the most damaging single group to remove, and it exists only in the wide table. Discarding the wide table would remove the report's clearest feature-level finding in exchange for a compute saving that this project, which retrains once, does not need.

What this experiment adds to the report's central claim. Section 6.6 shows that no learner is separable from another on pre-match data. This experiment shows that no *feature width* is separable either, across a more than threefold change in column count. Two independent axes of model-design freedom have now been varied and neither moves the pre-match result. That is considerably stronger evidence for the report's central position — that pre-match performance here is limited by the information available rather than by model capacity or feature engineering — than either result alone. It also sharpens the market comparison of Section 6.4: if tripling the feature count does not close the gap, the gap is unlikely to be a feature-count problem at all, and is better read as the information gap of team news and squad quality identified there.

Limitations of this experiment. It compares two specific feature sets, not width in general, and the narrow set is not a strict subset of the wide one — it retains pass completion and carries into the final third, which the wide set omits, so a small part of any difference is composition rather than width. Both runs share the same 344 test matches and the same single season, so the comparison inherits every external-validity limitation in Section 9.3. And a null result on 344 matches is a statement about detectable effects at this sample size, not a demonstration that the two are equivalent; a difference of the magnitude observed would require several times more test matches to resolve.

Other divergences between theory and measurement

- *Calibration does not always help.* Textbook treatment presents post-hoc calibration as a monotone improvement. On Task C it made the random forest worse, from 0.04257 to 0.08248 ECE, because isotonic regression overfits 277 validation rows. The theory assumes a calibration set large enough for a flexible non-parametric fit, and this project does not have one.
- *Kernel exponents do not match their asymptotic values.* Predicted exponents of 3.0 and 2.0 were measured as 2.291 and 1.938. At n below twenty thousand, fixed overheads and solver strategy switches dominate the asymptotic term, as discussed above.
- *More capacity did not produce more accuracy pre-match.* The tuned boosted ensembles, the kernel methods and the constant baseline all land within 0.01 RPS of one another on Task C. Standard model-selection intuition assumes the learner is the binding constraint; here the information content of the features is.
- *Oversampling improved recall and not the probabilistic score.* This is predictable from the definition of a proper scoring rule, but it is routinely presented in applied work as an unqualified improvement. Measuring both metrics is what makes the trade-off visible.

9. Conclusion and Limitations

9.1 Objectives achieved

All three model deliverables were formally defined, built and evaluated. Both feature pipelines run end to end from the raw JSON tree with leakage barriers enforced by assertions and unit tests rather than by claims in prose. Both papers were reimplemented from scratch and verified against their published equations and pseudocode by dedicated test suites. Every model was tuned under an equal candidate budget, calibrated on held-out validation data, and reported against both a constant baseline and the de-vigged market on identical matches. The complete metric suite is reported for every model on every task, including the many places where performance is poor.

9.2 Key findings

Live information matters far more than the choice of learner. The best in-play ranked probability score reaches 0.02266 at the final snapshot against 0.21862 for the frozen pre-match reference. No comparison between learners anywhere in this report comes close to that difference. The practical implication for a production system is that effort should go into what the model observes, not into which model observes it.

The pre-match problem is limited by information, not by capacity. Across every learner the spread on Task C runs from 0.21862 to 0.22812. A problem in which a constant baseline and a tuned gradient-boosted ensemble land that close together is one where the features, not the model, set the ceiling. The ablation supports this directly: only one feature group produces a clear deterioration when removed, and a substantial minority of deletions improve the metric.

Most of the leaderboard is not statistically supported. Of 87 pairwise comparisons, 21 survive Holm correction, and the survivors are concentrated on the in-play tasks. On Task C not a single comparison survives, including comparisons against the constant baseline. This is reported as a headline finding rather than buried, because the alternative would be to present an ordering the evidence does not support.

The market gap is real and is a feature gap. Zero of seven models beat the de-vigged market, which scores 0.19665 against 0.21878 for the best model. The market achieves this while never once assigning the draw the highest probability, which is the sharpest illustration in this report that probabilistic quality and arg-max accuracy are different objectives.

On P1. The reimplementations are faithful and reproduce the behavior its paper claims: it raises draw recall by more than any other arm tested, by 62 per cent relative to no treatment. It does not improve the probabilistic score, for a reason that follows from the definition of the metric rather than from any flaw in the method. Both halves of that result are reported.

On P2. Hierarchical shrinkage behaves as its paper predicts. It produces well-calibrated probabilities before any external calibration, because averaging shrunk leaf values pulls extreme estimates towards better-supported ancestors, and it selects its own shrinkage strength by cross-validation. It is consistently the most expensive model to fit, which is the price of that internal selection step. On pre-match data its advantage over simpler learners is not statistically detectable, which is a statement about the dataset rather than about the method.

9.3 Limitations

- ***One season.*** Roughly fifteen hundred matches from 2015/16 across four leagues. Because the split is chronological, the test block is the closing months of the

season, where the home-win rate is 48.55 per cent against 43.30 per cent in training. This is a genuine distribution shift and a further reason the models trail the market. It is nevertheless the correct way to split, since a random split would be optimistic.

- *Feature breadth.* Pass completion rate and carries *into* the final third are implemented in the shared aggregates module but are not called by either feature builder, so neither reaches the design matrices and no reported model uses them. Connecting them is a small change and is the cheapest available extension to the feature set. The pre-match table also contains nothing about squad quality or team news, which is the largest single component of the market gap.
- *Feature width is unresolved rather than optimal.* The pre-match matrix carries 231 columns against 896 training rows. The controlled comparison in Section 8E shows a narrower 75-column variant is not detectably different on any task, so the wide table is retained on point estimates and explanatory value rather than on demonstrated superiority. A null result at 344 test matches is not a demonstration of equivalence.
- *Correlated snapshots.* Nineteen snapshots per match are not nineteen independent observations. Grouped cross-validation and the match-clustered bootstrap handle this in tuning and testing, but the effective sample size for Task L remains the number of matches, not the number of rows. The worst-prediction analysis shows the same effect: ten worst Lc rows resolve to only two distinct matches.
- *Calibration data.* The validation split provides 277 matches, which is thin for isotonic regression. This is why the probability floor exists and why calibration occasionally makes the calibration error slightly worse. A per-phase calibrator for Model 3 would be preferable but would divide that thin data further still.
- *The market comparison is not like-for-like.* The market observes team news and money flow that these features cannot. It is the right reference point, but the gap should not be read purely as a difference in modelling quality.
- *Ablation power.* The feature-group ablation was run on the validation split with no multiplicity correction, and the two learners disagree in sign on many of the smaller entries. Individual rows are weak evidence; only the overall pattern is reported as a finding.

9.4 Future improvements

- Extend the width comparison of Section 8E beyond the two hand-built variants to a principled reduction — correlation pruning, variance thresholding, SHAP-ranked top-k and a principal-component projection — selected on validation and confirmed once on test. The two-point comparison run here cannot separate width from composition; a graded sweep could.

- Add squad-strength features from the lineup store, which is already built but currently unused by the feature pipelines. This is the most direct attack on the market gap.
- Re-extract the event store to retain pass outcome and carry end location, enabling the two missing quantities from the brief.
- Fit a per-phase calibrator for Model 3, which Section 7.2 shows is mis-specified at both ends of the match.
- Extend to further seasons, which would both increase the sample and allow the chronological split to hold out a whole season rather than the tail of one.

Appendix A. Hand Derivation of the P2 Core Equation

The P2 paper is Agarwal, Tan, Ronen, Singh and Yu, "Hierarchical Shrinkage: improving the accuracy and interpretability of tree-based models", ICML 2022, PMLR 162:111–135. Hierarchical shrinkage regularises an already-fitted tree by rewriting the value of each node as a damped sum along the path from the root to that node. It does not change the tree structure, only the values stored in it.

Step 1: the telescoping identity. Let the path from the root to a leaf be $t(0), t(1), \dots, t(L)$. Write $\mu(t)$ for the mean of the training responses falling in node t , and $N(t)$ for the number of training samples in t . The unshrunk prediction at the leaf is simply $\mu(t(L))$, which can be rewritten as a telescoping sum:

$$\mu(t_L) = \mu(t_0) + \sum_{l=1}^L [\mu(t_l) - \mu(t_{l-1})] \quad (6)$$

Every intermediate term in equation (6) cancels against its neighbour, leaving only $\mu(t(L))$. This is therefore an identity rather than an approximation. Its value is that it separates the prediction into a root-level estimate plus one increment per split, where each increment is the amount by which one additional split changed the estimate.

Step 2: the shrinkage. The key observation is that increments deep in the tree are estimated from few samples and are therefore unreliable, while increments near the root are estimated from many. Hierarchical shrinkage damps each increment in proportion to how little data supported it:

$$f_{HS}(x) = \mu(t_0) + \sum_{l=1}^L \frac{\mu(t_l) - \mu(t_{l-1})}{1 + \lambda/N(t_{l-1})} \quad (7)$$

Step 3: why the parent's count. The damping in equation (7) uses $N(t_{l-1})$, the parent's sample count, not the child's. This is deliberate. The increment is evidence about a split *of the parent*, so the relevant question is how much data was available when that split was chosen. Using the child's count would make the damping depend on the very partition whose reliability is in doubt, which is circular. The test suite verifies this by checking every root-to-leaf path of a fitted tree; substituting the child's count fails the test.

Step 4: limiting cases. Two limits check the form. As λ approaches zero, every damping factor $1/(1 + \lambda/N)$ approaches one and equation (7) collapses back to the telescoping identity of equation (6), recovering the unshrunk tree exactly. As λ grows without bound, every factor approaches zero and the prediction collapses to $\mu(t(0))$, the root mean, which is the constant model. So λ interpolates continuously between the fitted tree and a constant predictor. Both limits are asserted in the test suite rather than left as claims.

Step 5: the recursive form used in the implementation. The same statement can be written as a top-down recursion. Writing $v(t)$ for the shrunk value of node t and p for its parent:

$$v(t) = v(p) + \frac{\mu(t) - \mu(p)}{1 + \lambda/N(p)}, \quad v(t_0) = \mu(t_0) \quad (8)$$

Unrolling equation (8) from the root reproduces the closed form of equation (7) exactly. The implementation uses the recursion because it computes every node's shrunk value in a single pass over the tree, in time linear in the number of nodes, whereas evaluating the closed form independently per leaf would repeat work along shared prefixes. The test suite checks the equivalence on every root-to-leaf path of a fitted tree rather than on a single example, because an error in the recursion could easily be correct at one depth and wrong at another.

Step 6: the classification case. For classification, $\mu(t)$ is the vector of class proportions in node t , which lies on the probability simplex. The shrunk value is a combination of simplex points whose coefficients sum to one, so its coordinates also sum to one. Each coordinate stays within the unit interval because the damping factor lies between zero and one, so the update moves the child's proportion only part of the way from the parent's and cannot overshoot. The shrunk forest therefore produces valid probability vectors without renormalization, and the implementation clips only floating-point rounding error. This property is why the P2 model requires less external calibration than the boosted models, as observed in Section 7.1.

Step 7: selecting λ . λ is selected by cross-validation inside the training rows only, minimising log loss for classification and squared error for regression, so the validation split remains reserved for calibration. This is what makes a single fit of the P2 model cost several forest fits, which is visible in the compute table in Section 8D.

Appendix B. Reproducibility

B.1 Environment and dependencies

requirements.txt declares lower bounds rather than exact pins, so that the project installs on current Python releases; pinning old versions caused installation to fail on Python 3.14, where the pinned pandas has no prebuilt wheel and pip falls back to compiling from source. Reproducibility is preserved separately: the pipeline runner records the exact resolved versions at run time into 00_environment.txt and writes requirements.lock.txt. Installing that lock file reproduces the environment that produced the results in this report, while ordinary users install the unpinned file.

Table 37. Resolved package versions for the run that produced every number in this report, read from the environment record written at run time.

Package	Version	Package	Version
Python	3.14.0 (CPython)	shap	0.52.0
numpy	2.3.4	matplotlib	3.10.7
pandas	2.3.3	pillow	11.3.0
scipy	1.16.3	plotly	6.9.0
scikit-learn	1.8.0	psutil	7.2.2
xgboost	3.3.0	reportlab	5.0.0
lightgbm	4.7.0	Operating system	Windows 11 (AMD64)
imbalanced-learn	0.14.2	Run date	2026-08-19

B.2 Random seeds and fixed configuration

Table 38. Fixed configuration and seeds. The search seed is offset by a checksum of the model and task name rather than by Python's built-in hash, because string hashing is randomised per process by default and would make the search irreproducible across runs.

Setting	Value	Where it applies
Global random state	0	every estimator, resampler and split
Search seed offset	checksum of model and task name	hyperparameter search
Seed-repetition states	3 states	stability diagnostic in Section 6.6
Bootstrap resamples	clustered on match identity	significance testing in Section 6.6
Rolling form window	5 matches, lagged by 1	pre-match feature builder
Snapshot minutes	$t = 0, 5, \dots, 90$ (19 per match)	in-play feature builder
Recent in-play window	10 minutes	in-play feature builder
Margin clip	$[-5, +5]$	label store
Cross-validation folds	3, grouped by match on Task L	hyperparameter search, λ selection
Candidate budget	12 per model per task	hyperparameter search
Calibration method	isotonic, fitted on validation	all probabilistic classifiers
λ grid for P2	$\{0, 0.1, 1, 10, 25, 50, 100\}$	hierarchical shrinkage

Table 39. Selected hyperparameter configurations for every tuned model on every task.

Task	Model	Selected configuration
C	gbm	l2_regularization=0.1, learning_rate=0.02, max_iter=200, max_leaf_nodes=63, min_samples_leaf=50
C	kernel_svm	C=0.5, gamma=scale
C	lightgbm	colsample_bytree=0.6, learning_rate=0.02, n_estimators=400, num_leaves=15, reg_lambda=5.0, subsample=0.6
C	p2_hier_shrinkage	max_depth=8, n_estimators=200
C	random_forest	max_depth=None, max_features=0.3, min_samples_leaf=25, n_estimators=500
C	xgboost	colsample_bytree=0.6, learning_rate=0.02, max_depth=3, n_estimators=300, reg_lambda=1.0, subsample=0.8
R	gbm	l2_regularization=1.0, learning_rate=0.02, max_iter=200, max_leaf_nodes=15, min_samples_leaf=10
R	kernel_ridge_exact	alpha=3.0, gamma=None
R	kernel_ridge_nystroem	alpha=3.0, gamma=None, n_components=250
R	kernel_svr	C=0.5, epsilon=0.3, gamma=scale
R	lightgbm	colsample_bytree=0.8, learning_rate=0.02, n_estimators=200, num_leaves=31, reg_lambda=1.0, subsample=0.6
R	p2_hier_shrinkage	max_depth=8, n_estimators=500
R	random_forest	max_depth=None, max_features=1.0, min_samples_leaf=10, n_estimators=300
R	xgboost	colsample_bytree=0.6, learning_rate=0.02, max_depth=3, n_estimators=200, reg_lambda=5.0, subsample=0.6
Lc	gbm	l2_regularization=1.0, learning_rate=0.02, max_iter=200, max_leaf_nodes=63, min_samples_leaf=20
Lc	lightgbm	colsample_bytree=0.8, learning_rate=0.02, n_estimators=200, num_leaves=31, reg_lambda=1.0, subsample=0.8
Lc	p2_hier_shrinkage	max_depth=8, n_estimators=500
Lc	random_forest	max_depth=14, max_features=0.3, min_samples_leaf=25, n_estimators=300
Lc	xgboost	colsample_bytree=0.6, learning_rate=0.02, max_depth=4, n_estimators=200, reg_lambda=5.0, subsample=1.0
Lr	gbm	l2_regularization=0.0, learning_rate=0.02, max_iter=200, max_leaf_nodes=63, min_samples_leaf=20
Lr	lightgbm	colsample_bytree=0.8, learning_rate=0.02, n_estimators=600, num_leaves=63, reg_lambda=5.0, subsample=0.8
Lr	p2_hier_shrinkage	max_depth=8, n_estimators=500
Lr	random_forest	max_depth=14, max_features=0.3, min_samples_leaf=25, n_estimators=200
Lr	xgboost	colsample_bytree=0.6, learning_rate=0.02, max_depth=4, n_estimators=300, reg_lambda=5.0, subsample=1.0

B.3 Pipeline stages and how to reproduce

The stages form a dependency chain driven by `run_pipeline.py`, which runs them in order and captures the console output of each into a per-step log. Each stage writes its outputs to disk and the next stage reads them, so a stage can be re-run in isolation once its inputs exist. The order is: relational store construction, cleaning, temporal splits, market baseline, pre-match features, in-play features, hyperparameter search, model training, experiments (market comparison, in-play curves, kernel scaling, compute profile, margin conversion, significance, ablation), SHAP analysis, visualisation, and report build. Tuning saves `best_params.json` after each task, so an interrupted search resumes rather than restarting.

B.4 Version control history

Development history was exported with `git log --oneline --all`. The repository contains 102 commits across 18 branches, spanning 7 July 2026 to 25 August 2026, with feature work developed on topic branches and merged back to main. The fifty most recent commits are listed below.

```
201df5f Merge branch 'docs/report-prose-synce'
e3f4e6d Merge branch 'feat/prediction-service'
284de98 docs(report): align section 3 and 8 prose with the implemented pipeline
1111821 feat(service): FastAPI prediction service, latency benchmark, live dashboard
f8ade9f feat(ablation): validation-only ablation over feature groups and snapshot frequency
7d1eae5 feat(features): widen both pipelines to the brief's 2.2/5A/5B list
846966b fix(run_pipeline): unwrap set-in-set literal in needs_store preflight
60cbb80 [Refactor] 1. Removed all docstrings and comments except section dividers and run instru
b37d71e Merge branch 'fix/tuning-resume' Merge fix/tuning-resume
1433e8d ✖ Function ✖ 1. Modified 'main' in run_models.py to accept a 'TASKS' env var
b076aa4 ✖ Function ✖ 1. Added 'save' Function to tuning.py to persist tuning_result
5c950be Merge branch 'fix/environment-portability' Merge fix/environment-portability
ceba9e5 🍌 Dependencies 🍌 1. Modified requirements.txt to use lower-bound versions instead
f4e5816 Merge branch 'feature/pipeline-runner' Merge feature/pipeline-runner
0a3f7c5 🔄 Progress Monitor 🔄 1. Added 'check_progress.py' to check whether the pipeline is
2d84278 ✖ Pipeline Runner ✖ 1. Added 'run_pipeline.py' to run the full remaining pip
5394d76 ✖ Console Capture ✖ 1. Added 'capture_console.py' to run pipeline steps and
81994d7 Merge branch 'feature/prematch-tests' Merge feature/prematch-tests
be9280b 🟢 Tests 🟢 1. Added test_prematch_features.py for the pre-match feature builder
4932716 Merge branch 'feature/generated-report' Merge feature/generated-report
9b31942 [Report Build] 1. Added render_docx.js script 1.1. Reads report_content.json (wr
12c75b5 📄 Docs 📄 1. Updated README.md documenting the two-format report build (PDF then DO
8922248 Merge branch 'fix/nystroem-landmark-clamp' Merge fix/nystroem-landmark-clamp
e77d61b [Backend] 1. Fixed kernel_ridge_nystroem in model_zoo.py to clamp n_components to
a0e1ebf [Models] 1. Fixed snapshot_frequency_ablation in ablation.py to only thin TRAINING
0f8631a [Models] 1. Added ablation.py script 1.1. Validation-split only - test split nev
01b6a47 [Models] 1. Added compute_profile.py script 1.1. Measures fit time, peak fit mem
fa78673 [Models] 1. Clarified significance.py docstring on why seed repetition measures s
f8b6c2e [Models] 1. Added seed-repetition testing to significance.py 1.1. Added N_SEEDS
5a5e60f [Models] 1. Added significance.py script 1.1. Implemented match-clustered bootst
34fb7e7 [Models] 1. Added margin_to_probability.py script 1.1. Implemented ordinal thres
b1459d0 Merge feature/explainability
c9f3892 fix(tuning): pass resampling=, not the removed use_p1 keyword
81f4ad3 feat(shap): in-play attribution timeline across one full match
216a953 feat(shap): adjudicate the ten worst predictions per task
ac6fc44 feat(shap): global and local attributions per task
0fcbbe5 feat(p2): materialise shrunk node values for TreeSHAP
269bc3e feat(modeling_common): return feature names, the fitted transform and column lists
232830e fix(tuning): pass resampling=, not the removed use_p1 keyword
ec0bbe3 Revise repository structure in README
075a79b - Reorganized the project - Updated README.md
3a43680 feat(index.html): slides for presentation
f6c976d Merge origin/main: paper PDFs renamed, P2 paper added
a051859 Merge PR 6: chore/pipeline-and-defence-pack
7fbd19e Merge PR 5: feat/md2-required-analyses
c4c26c1 Merge PR 4: feat/model-training-harness
eca3d55 Merge PR 3: feat/p2-hierarchical-shrinkage
10a86e4 Merge PR 2: docs/p2-paper-selection
6f5032a Merge PR 1: fix/env-pins-and-inplay-cut
7945e79 docs(forME): keep the original project audit as a historical reference
```

Appendix C. Complete Result Tables

The tables below carry the full per-row results underlying the summaries in the body of the report. They are placed here rather than in the main text so that the body remains readable, but nothing is omitted: every model, every arm, every phase, every seed and every ablation configuration appears. Each table is referenced from the section that discusses it.

Table 40. Every pairwise comparison that survives Holm correction, with its bootstrap confidence interval. All twenty-one are concentrated on the tasks where a genuinely large effect exists.

Task	Model A	Model B	Mean difference	CI low	CI high	P (Holm)	Verdict
R	dummy	kernel_ridge_exact	+0.07866	+0.03330	+0.12206	0.0180	kernel_ridge_exact better
R	dummy	kernel_svr	+0.07979	+0.03579	+0.12283	0.0180	kernel_svr better
R	dummy	p2_hier_shrinkage	+0.12345	+0.05697	+0.18932	0.0180	p2_hier_shrinkage better
R	dummy	kernel_ridge_nystroem	+0.07463	+0.03178	+0.11655	0.0330	kernel_ridge_nystroem better
R	dummy	xgboost	+0.12105	+0.05264	+0.19092	0.0330	xgboost better
R	dummy	random_forest	+0.12344	+0.05595	+0.19217	0.0465	random_forest better
Lc	dummy	gbm	+0.07076	+0.05688	+0.08457	0.0000	gbm better
Lc	dummy	lightgbm	+0.07446	+0.06003	+0.08815	0.0000	lightgbm better
Lc	dummy	p2_hier_shrinkage	+0.07802	+0.06331	+0.09242	0.0000	p2_hier_shrinkage better
Lc	dummy	random_forest	+0.08139	+0.06756	+0.09510	0.0000	random_forest better
Lc	dummy	xgboost	+0.07796	+0.06353	+0.09188	0.0000	xgboost better
Lc	gbm	random_forest	+0.01064	+0.00431	+0.01680	0.0100	random_forest better
Lr	dummy	gbm	+0.46727	+0.37448	+0.55973	0.0000	gbm better
Lr	dummy	lightgbm	+0.46729	+0.37967	+0.55570	0.0000	lightgbm better
Lr	dummy	p2_hier_shrinkage	+0.50904	+0.41501	+0.60378	0.0000	p2_hier_shrinkage better
Lr	dummy	random_forest	+0.51909	+0.43021	+0.61023	0.0000	random_forest better
Lr	dummy	xgboost	+0.48225	+0.39578	+0.56928	0.0000	xgboost better
Lr	gbm	random_forest	+0.05182	+0.03011	+0.07333	0.0000	random_forest better
Lr	lightgbm	random_forest	+0.05181	+0.02381	+0.07859	0.0000	random_forest better
Lr	random_forest	xgboost	-0.03684	-0.05862	-0.01576	0.0080	random_forest better
Lr	gbm	p2_hier_shrinkage	+0.04177	+0.01294	+0.07158	0.0315	p2_hier_shrinkage better

Table 41. Test metric across random seeds. This is a stability diagnostic only. A near-zero standard deviation means the learner is reproducible when reseeded; it does not mean its advantage over another learner is real.

Task	Model	Metric	Mean	Std. dev.	Min	Max
C	dummy	rps	0.22817	0.000000	0.22817	0.22817
C	gbm	rps	0.22669	0.000000	0.22669	0.22669
C	kernel_svm	rps	0.21976	0.000195	0.21962	0.21998
C	lightgbm	rps	0.22326	0.000955	0.22254	0.22434
C	p2_hier_shrinkage	rps	0.21048	0.000236	0.21021	0.21062
C	random_forest	rps	0.20923	0.000547	0.20863	0.20969
C	xgboost	rps	0.21435	0.001132	0.21308	0.21527
R	dummy	mae	1.48997	0.000000	1.48997	1.48997
R	gbm	mae	1.39335	0.000000	1.39335	1.39335
R	kernel_ridge_exact	mae	1.41131	0.000000	1.41131	1.41131
R	kernel_ridge_nystroem	mae	1.41722	0.003438	1.41514	1.42119
R	kernel_svr	mae	1.41018	0.000000	1.41018	1.41018
R	lightgbm	mae	1.38006	0.007751	1.37115	1.38523
R	p2_hier_shrinkage	mae	1.36464	0.003231	1.36091	1.36651
R	random_forest	mae	1.36413	0.004137	1.35936	1.36653
R	xgboost	mae	1.36804	0.002713	1.36500	1.37021
Lc	dummy	rps	0.22817	0.000000	0.22817	0.22817
Lc	gbm	rps	0.16751	0.002270	0.16524	0.16978
Lc	lightgbm	rps	0.15270	0.000528	0.15233	0.15331
Lc	p2_hier_shrinkage	rps	0.15396	0.000545	0.15334	0.15438
Lc	random_forest	rps	0.14389	0.000764	0.14305	0.14454
Lc	xgboost	rps	0.14649	0.000515	0.14596	0.14698
Lr	dummy	mae	1.48997	0.000000	1.48997	1.48997
Lr	gbm	mae	1.01409	0.007533	1.00871	1.02270
Lr	lightgbm	mae	1.02919	0.008193	1.02268	1.03839
Lr	p2_hier_shrinkage	mae	0.97987	0.000947	0.97909	0.98092
Lr	random_forest	mae	0.96695	0.004246	0.96244	0.97087
Lr	xgboost	mae	0.99931	0.007380	0.99389	1.00771

Table 42. Per-model, per-phase calibration for Model 3. The sign change in over-confidence between the opening and closing phases is present for every learner.

Model	Phase	ECE	Mean confidence	Accuracy	Over-confidence	RPS
gbm	0-15	0.09362	0.57421	0.48159	+0.09262	0.22218
gbm	15-30	0.09068	0.59016	0.51163	+0.07853	0.20689
gbm	30-45	0.06925	0.60412	0.54942	+0.05470	0.18499
gbm	45-60	0.03901	0.64013	0.63566	+0.00447	0.15267
gbm	60-75	0.06433	0.66680	0.71318	-0.04638	0.12383
gbm	75-90	0.11311	0.70157	0.81468	-0.11311	0.07957
lightgbm	0-15	0.10309	0.58269	0.48643	+0.09626	0.22227
lightgbm	15-30	0.09565	0.60377	0.51357	+0.09021	0.20826
lightgbm	30-45	0.05970	0.62568	0.57752	+0.04816	0.18139
lightgbm	45-60	0.03223	0.66495	0.64729	+0.01766	0.14540
lightgbm	60-75	0.03168	0.69742	0.70349	-0.00607	0.11792
lightgbm	75-90	0.09527	0.73167	0.82340	-0.09173	0.07348
p2_hier_shrinkage	0-15	0.08460	0.57119	0.48934	+0.08185	0.21643
p2_hier_shrinkage	15-30	0.11682	0.62166	0.50484	+0.11682	0.20663

p2_hier_shrinkage	30-45	0.10557	0.65531	0.55233	+0.10299	0.18460
p2_hier_shrinkage	45-60	0.08597	0.71397	0.62888	+0.08510	0.14628
p2_hier_shrinkage	60-75	0.03976	0.75746	0.72578	+0.03169	0.11340
p2_hier_shrinkage	75-90	0.09171	0.78538	0.87573	-0.09035	0.06247
random_forest	0-15	0.09549	0.57355	0.49709	+0.07646	0.21835
random_forest	15-30	0.09747	0.61034	0.51453	+0.09581	0.20441
random_forest	30-45	0.06648	0.64303	0.57655	+0.06648	0.18167
random_forest	45-60	0.06520	0.70894	0.65407	+0.05487	0.14395
random_forest	60-75	0.07220	0.77586	0.72674	+0.04911	0.11107
random_forest	75-90	0.05172	0.83937	0.88953	-0.05017	0.05238
xgboost	0-15	0.10683	0.57284	0.46609	+0.10676	0.22691
xgboost	15-30	0.10725	0.61403	0.50678	+0.10725	0.21216
xgboost	30-45	0.08331	0.65308	0.56977	+0.08331	0.18288
xgboost	45-60	0.09094	0.71838	0.63566	+0.08273	0.14411
xgboost	60-75	0.05825	0.77807	0.72190	+0.05617	0.11092
xgboost	75-90	0.05563	0.82812	0.87427	-0.04615	0.05553

Table 43. The forty worst predictions in full, ten per task, with the model's probability for the realised outcome beside the market's, which is what the adjudication compares.

Task	Rank	Match	Minute	Loss	Actual	Model p	Market p	Verdict
C	1	266557		0.69683	A	0.09199	0.05618	inherently uncertain
C	2	3901193		0.62562	A	0.12067	0.09507	inherently uncertain
C	3	3754124		0.61240	A	0.13118	0.20566	model error
C	4	3879803		0.60692	H	0.14288	0.14632	inherently uncertain
C	5	3879792		0.57291	A	0.15272	0.21188	model error
C	6	3825841		0.55632	A	0.15714	0.15748	inherently uncertain
C	7	3879806		0.54899	A	0.16135	0.21195	model error
C	8	267533		0.54348	A	0.15446	0.17466	inherently uncertain
C	9	3901185		0.54139	A	0.15481	0.27140	model error
C	10	3901221		0.51969	A	0.17816	0.23779	model error
R	1	3879847		4.79398	-4.0			inherently uncertain
R	2	3901221		4.06382	-3.0			inherently uncertain
R	3	3754333		4.04514	4.0			inherently uncertain
R	4	3901187		3.97531	4.0			inherently uncertain
R	5	3754075		3.95925	-4.0			inherently uncertain
R	6	266310		3.95386	-5.0			inherently uncertain
R	7	3901193		3.93535	-2.0			model error
R	8	3825871		3.93276	4.0			inherently uncertain
R	9	3901206		3.93002	4.0			inherently uncertain
R	10	3825896		3.90453	4.0			inherently uncertain
Lc	1	3754064	45	0.93394	H	0.00566	0.37220	noisy observation
Lc	2	3754064	40	0.91822	H	0.00972	0.37220	noisy observation
Lc	3	3754064	60	0.91106	H	0.00714	0.37220	noisy observation
Lc	4	3754064	55	0.90400	H	0.01045	0.37220	noisy observation
Lc	5	3754064	35	0.90274	H	0.01472	0.37220	noisy observation
Lc	6	3754064	50	0.90103	H	0.01025	0.37220	noisy observation
Lc	7	3754064	30	0.89659	H	0.01544	0.37220	noisy observation

Lc	8	3754064	80	0.89103	H	0.01160	0.37220	noisy observation
Lc	9	3879846	70	0.88887	H	0.01888	0.19062	noisy observation
Lc	10	3754064	75	0.88509	H	0.01195	0.37220	noisy observation
Lr	1	3879847	10	4.79265	-4.0			inherently uncertain
Lr	2	3879847	0	4.77245	-4.0			inherently uncertain
Lr	3	3879847	5	4.72708	-4.0			inherently uncertain
Lr	4	3879835	5	4.53736	4.0			noisy observation
Lr	5	3879835	10	4.45956	4.0			noisy observation
Lr	6	3879835	15	4.43592	4.0			noisy observation
Lr	7	3879835	20	4.40035	4.0			noisy observation
Lr	8	3879835	25	4.32772	4.0			noisy observation
Lr	9	3825871	0	4.28144	4.0			inherently uncertain
Lr	10	3879835	30	4.27424	4.0			noisy observation

Table 44. Per-learner results for all six arms, showing that the recall ordering is consistent across learners rather than driven by one of them.

Arm	Model	RPS	Draw precision	Draw recall	Draw F1	ECE
vanilla	gbm	0.21862	0.2258	0.0886	0.1273	0.06405
vanilla	kernel_svm	0.22482	0.1538	0.0253	0.0435	0.01438
vanilla	lightgbm	0.21891	0.1935	0.0760	0.1091	0.06694
vanilla	random_forest	0.21962	0.2373	0.1772	0.2029	0.08248
smote	gbm	0.21637	0.2821	0.1392	0.1864	0.05059
smote	kernel_svm	0.22335	0.2553	0.1519	0.1905	0.06876
smote	lightgbm	0.21903	0.1600	0.1013	0.1240	0.08538
smote	random_forest	0.21696	0.1698	0.1139	0.1364	0.09778
borderline_smote	gbm	0.22100	0.1667	0.0760	0.1043	0.06345
borderline_smote	kernel_svm	0.22142	0.2983	0.2152	0.2500	0.04777
borderline_smote	lightgbm	0.21841	0.2000	0.1013	0.1344	0.04738
borderline_smote	random_forest	0.21536	0.2391	0.1392	0.1760	0.06225
adasyn	gbm	0.22139	0.2273	0.0633	0.0990	0.04952
adasyn	kernel_svm	0.22305	0.2692	0.0886	0.1333	0.02839
adasyn	lightgbm	0.22209	0.1600	0.1013	0.1240	0.09211
adasyn	random_forest	0.21545	0.1316	0.0633	0.0855	0.09072
class_weight	gbm	0.22078	0.2000	0.0633	0.0961	0.03190
class_weight	kernel_svm	0.21909	0.2889	0.1646	0.2097	0.02324
class_weight	lightgbm	0.22036	0.2381	0.1266	0.1653	0.05471
class_weight	random_forest	0.21851	0.1538	0.0760	0.1017	0.07420
p1_gsmotenc	gbm	0.22196	0.0769	0.0253	0.0381	0.03571
p1_gsmotenc	kernel_svm	0.22494	0.2222	0.2278	0.2250	0.06720
p1_gsmotenc	lightgbm	0.22047	0.1837	0.1139	0.1406	0.05722
p1_gsmotenc	random_forest	0.22215	0.2400	0.2278	0.2338	0.06666

Table 45. Raw scaling measurements. The theoretical Gram column is $8n^2$ bytes expressed in megabytes and is independent of the measurement; comparing it against peak memory shows how closely each method tracks the dense-matrix bound.

Method	Training rows	Fit seconds	Peak memory (MB)	Theoretical Gram (MB)
kernel_ridge_exact	500	0.0458	5.8	1.9
kernel_ridge_exact	1,000	0.0584	19.0	7.6
kernel_ridge_exact	2,000	0.2247	66.7	30.5
kernel_ridge_exact	4,000	1.0178	254.5	122.1
kernel_ridge_exact	8,000	4.7970	1002.1	488.3
kernel_ridge_exact	12,000	42.1524	2218.1	1098.6
kernel_ridge_exact	17,024	114.9797	4450.0	2211.1
kernel_svr_exact	500	0.0885	1.8	1.9
kernel_svr_exact	1,000	0.3512	2.3	7.6
kernel_svr_exact	2,000	1.2312	13.7	30.5
kernel_svr_exact	4,000	4.4512	51.1	122.1
kernel_svr_exact	8,000	20.8669	178.9	488.3
kernel_svr_exact	12,000	44.6097	201.1	1098.6
kernel_svr_exact	17,024	76.1950	203.6	2211.1
kernel_ridge_nystroem	500	0.1404	9.9	1.9
kernel_ridge_nystroem	1,000	0.1527	14.9	7.6
kernel_ridge_nystroem	2,000	0.3029	22.5	30.5
kernel_ridge_nystroem	4,000	0.2296	37.9	122.1
kernel_ridge_nystroem	8,000	0.2946	68.3	488.3
kernel_ridge_nystroem	12,000	0.3385	97.3	1098.6
kernel_ridge_nystroem	17,024	0.4113	138.0	2211.1

Table 46. Task C: change in validation ranked probability score when a feature group is dropped, relative to the full feature set (random forest 0.19547, XGBoost 0.19682). A positive value means the drop made things worse, so the group carried signal.

Configuration	Random forest	XGBoost
drop_venue_form	+0.00305	+0.00154
drop_agg_shooting	+0.00135	+0.00205
drop_form_goals	+0.00053	+0.00132
drop_agg_scoring	+0.00046	+0.00165
drop_agg_passing	+0.00042	+0.00017
drop_form_xg	+0.00035	+0.00037
drop_schedule	+0.00013	-0.00071
drop_league	+0.00003	+0.00075
drop_form_results	+0.00003	+0.00111
drop_agg_discipline	-0.00014	+0.00085
drop_head_to_head	-0.00022	-0.00083
drop_agg_pressure	-0.00035	+0.00326

drop_agg_possession	-0.00051	+0.00075
drop_agg_defence	-0.00060	+0.00004
drop_agg_territory	-0.00066	-0.00075
drop_agg_set_pieces	-0.00078	+0.00072

Table 47. Task R: change in validation mean absolute error when a feature group is dropped, relative to the full feature set (random forest 1.21489, XGBoost 1.21099).

Configuration	Random forest	XGBoost
drop_venue_form	+0.00501	+0.00905
drop_league	+0.00131	+0.00808
drop_schedule	+0.00127	+0.00657
drop_agg_possession	+0.00103	+0.00084
drop_head_to_head	+0.00099	-0.00049
drop_agg_scoring	+0.00097	-0.00151
drop_agg_discipline	+0.00071	-0.00164
drop_form_xg	+0.00068	+0.00183
drop_form_goals	+0.00062	-0.00013
drop_agg_set_pieces	-0.00045	-0.00038
drop_form_results	-0.00072	+0.00059
drop_agg_defence	-0.00259	-0.01022
drop_agg_shooting	-0.00347	+0.00209
drop_agg_pressure	-0.00777	+0.00667
drop_agg_territory	-0.00870	-0.01115
drop_agg_passing	-0.01276	-0.00367

Table 48. Task Lc: change in validation ranked probability score when a feature group is dropped, relative to the full feature set (random forest 0.12902, XGBoost 0.12985). The prematch_only arm removes every in-play feature at once.

Configuration	Random forest	XGBoost
prematch_only	+0.07103	+0.07067
drop_inplay_score	+0.04964	+0.04889
drop_venue_form	+0.00118	+0.00178
drop_agg_scoring	+0.00073	+0.00070
drop_form_results	+0.00059	+0.00121
drop_agg_set_pieces	+0.00051	-0.00027
drop_form_xg	+0.00047	+0.00076
drop_form_goals	+0.00033	+0.00133
drop_inplay_possession	+0.00033	+0.00010
drop_inplay_discipline	+0.00028	+0.00032
drop_agg_possession	+0.00023	+0.00099
drop_agg_passing	+0.00020	-0.00081

drop_agg_defence	+0.00015	+0.00093
drop_head_to_head	+0.00014	+0.00033
drop_agg_shooting	-0.00007	+0.00001
drop_agg_pressure	-0.00009	+0.00062
drop_inplay_tempo	-0.00010	+0.00029
drop_agg_discipline	-0.00011	+0.00034
drop_inplay_defence	-0.00012	+0.00110
drop_schedule	-0.00012	+0.00078
drop_inplay_territory	-0.00014	+0.00084
drop_inplay_passing	-0.00030	+0.00044
drop_league	-0.00032	+0.00043
drop_inplay_pressure	-0.00047	+0.00059
drop_inplay_shots	-0.00055	+0.00045
drop_agg_territory	-0.00098	+0.00036
drop_inplay_xg	-0.00122	+0.00051

Table 49. Task Lr: change in validation mean absolute error when a feature group is dropped, relative to the full feature set (random forest 0.83096, XGBoost 0.86541).

Configuration	Random forest	XGBoost
prematch_only	+0.39839	+0.37025
drop_inplay_score	+0.26796	+0.23378
drop_inplay_xg	+0.01359	-0.00172
drop_form_goals	+0.01032	-0.00016
drop_agg_shooting	+0.00925	+0.00373
drop_inplay_territory	+0.00763	-0.00136
drop_inplay_pressure	+0.00666	-0.00167
drop_agg_set_pieces	+0.00564	+0.00073
drop_form_results	+0.00541	+0.00012
drop_inplay_possession	+0.00528	-0.00050
drop_head_to_head	+0.00425	-0.00538
drop_form_xg	+0.00390	-0.00067
drop_agg_scoring	+0.00353	+0.00193
drop_inplay_shots	+0.00348	+0.00496
drop_venue_form	+0.00340	-0.00300
drop_agg_defence	+0.00314	-0.00021
drop_inplay_discipline	+0.00263	-0.00237
drop_inplay_tempo	+0.00261	+0.00119
drop_agg_pressure	+0.00193	-0.00335
drop_inplay_passing	+0.00179	-0.00377
drop_agg_discipline	+0.00156	+0.00498
drop_agg_passing	+0.00118	-0.00831
drop_inplay_defence	+0.00090	+0.00654
drop_schedule	+0.00007	+0.00203
drop_league	-0.00120	-0.00163
drop_agg_possession	-0.00230	-0.00046
drop_agg_territory	-0.00766	+0.00017

References

- [1] StatsBomb / Hudl StatsBomb (2018–2025). *StatsBomb Open Data*. Available at: <https://github.com/statsbomb/open-data>. Free for research and non-commercial use with attribution. Data provided by Hudl StatsBomb.
- [2] Football-Data.co.uk. *Historical Football Results and Betting Odds Data*. Available at: <https://www.football-data.co.uk>.
- [3] Fonseca, J. and Bacao, F. (2023). Geometric SMOTE for imbalanced datasets with nominal and continuous features. *Expert Systems with Applications*, 234, 121053.
- [4] Agarwal, A., Tan, Y. S., Ronen, O., Singh, C. and Yu, B. (2022). Hierarchical Shrinkage: Improving the accuracy and interpretability of tree-based models. *Proceedings of the 39th International Conference on Machine Learning*, PMLR 162:111–135.
- [5] Chawla, N. V., Bowyer, K. W., Hall, L. O. and Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- [6] Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- [7] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T. Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30, 3146–3154.
- [8] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [9] Lundberg, S. M. and Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774.
- [10] Lundberg, S. M. et al. (2020). From Local Explanations to Global Understanding with Explainable AI for Trees. *Nature Machine Intelligence*, 2(1), 56–67.
- [11] Lemaitre, G., Nogueira, F. and Aridas, C. K. (2017). Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning. *Journal of Machine Learning Research*, 18(17), 1–5.
- [12] Constantinou, A. C. and Fenton, N. E. (2012). Solving the Problem of Inadequate Scoring Rules for Assessing Probabilistic Football Forecast Models. *Journal of Quantitative Analysis in Sports*, 8(1).
- [13] Niculescu-Mizil, A. and Caruana, R. (2005). Predicting Good Probabilities with Supervised Learning. *Proceedings of the 22nd International Conference on Machine Learning*, 625–632.
- [14] Holm, S. (1979). A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, 6(2), 65–70.
- [15] Williams, C. K. I. and Seeger, M. (2001). Using the Nyström Method to Speed Up Kernel Machines. *Advances in Neural Information Processing Systems*, 13, 682–688.

List of Tables

Every entry links to the table it names. Page numbers refer to the printed pagination.

Table 1. Competition-seasons retained for modelling.....	6
Table 2. Reasons for exclusion of rejected competition-seasons	6
Table 3. Alias-map resolution methods across the eighty club-name pairs	7
Table 4. Representative alias-map entries requiring fuzzy matching or manual override	7
Table 5. Odds tagging coverage by league over the full dataset	8
Table 6. Every match excluded from the odds-tagged set, with its reason	8
Table 7. Aggregated data-quality log over the cleaned store	8
Table 8. Chronological split of the 1,517 matches, with outcome shares	9
Table 9. Hard-filter audit for both selected papers.....	14
Table 10. P1 fidelity table: G-SMOTENC components, implementation and tests.....	15
Table 11. P2 fidelity table: hierarchical shrinkage properties, implementation and tests	16
Table 12. The model zoo: every learner, the tasks it covers and its role.....	18
Table 13. Hyperparameter search budgets and cross-validated scores per task.....	19
Table 14. Task C probabilistic results on the test split.....	22
Table 15. Task C per-class precision, recall and F1 on the test split.....	22
Table 16. Task R results on the test split, ranked by mean absolute error	23
Table 17. Task Lc probabilistic results, aggregated over all snapshot minutes	24
Table 18. Task Lc per-class precision, recall and F1 across all snapshots.....	24
Table 19. Task Lr results, aggregated over all snapshot minutes.....	24
Table 20. Odds tagging coverage of the test split by competition	25
Table 21. Task C models against the de-vigged market on 343 test matches	25
Table 22. Margin-to-probability conversion against the directly trained classifier.....	27
Table 23. Match-clustered bootstrap results by task, after Holm–Bonferroni correction	28
Table 24. Model 3 calibration by game phase.....	30
Table 25. Mean absolute SHAP value per feature, the six largest per task.....	32
Table 26. Adjudication of the ten worst predictions per task.....	36
Table 27. Task Lc ranked probability score at seven representative minutes	38
Table 28. Task Lr mean absolute error at seven representative minutes.....	39
Table 29. Six-arm imbalance comparison on the pre-match classification task.....	42
Table 30. P1 against no resampling on the validation split.....	43
Table 31. Empirical against theoretical scaling exponents	44
Table 32. Compute and memory comparison across all models and tasks	45
Table 33. Snapshot frequency ablation with a fixed evaluation set	48
Table 34. Feature-set width comparison, best learner per task	50
Table 35. Per-learner paired comparison across the two feature widths	50

Table 36. Draw-class detection across the two feature widths.....	51
Table 37. Resolved package versions for the run behind every reported number.....	58
Table 38. Fixed configuration and random seeds.....	58
Table 39. Selected hyperparameter configurations for every tuned model and task.....	59
Table 40. Every pairwise comparison that survives Holm correction.....	61
Table 41. Test metric across random seeds, reported as a stability diagnostic	62
Table 42. Per-model, per-phase calibration for Model 3.....	62
Table 43. The forty worst predictions in full, ten per task	63
Table 44. Per-learner results for all six imbalance arms	64
Table 45. Raw scaling measurements against the dense-matrix memory bound	65
Table 46. Task C feature-group ablation on the validation split	65
Table 47. Task R feature-group ablation on the validation split	66
Table 48. Task Lc feature-group ablation on the validation split.....	66
Table 49. Task Lr feature-group ablation on the validation split	67

[Return to Contents](#)

List of Figures

Every entry links to the figure it names. Page numbers refer to the printed pagination.

Figure 1. Outcome shares by split.....	9
Figure 2. Task C reliability diagrams, raw and after isotonic calibration	29
Figure 3. Task Lc reliability diagrams, raw and after isotonic calibration	29
Figure 4. Global SHAP beeswarm for Task C, random forest.....	33
Figure 5. Global SHAP beeswarm for Task R, P2 hierarchical shrinkage	33
Figure 6. Global SHAP beeswarm for Task Lc, random forest	34
Figure 7. Global SHAP beeswarm for Task Lr, random forest.....	34
Figure 8. SHAP waterfall for the worst Task C failure judged a model error	36
Figure 9. Ranked probability score against match minute.....	38
Figure 10. Mean absolute margin error against match minute.....	39
Figure 11. SHAP timeline across one complete match.....	40
Figure 12. Fit time against training rows on logarithmic axes.....	44

[Return to Contents](#)